German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026

Milestone 2: Ride Hailing (Uber) Platform

Polyglot Persistence, Authentication, Caching & Design Patterns

Deadline is Saturday 02/05/2026 at 11:59 PM

Contents

1 Overview

This milestone is worth **15%** of your final grade. You will extend the Ride Hailing Platform you built in Milestone 1 by adding **authentication**, **five NoSQL databases**, **caching**, and **design patterns**. Your existing 5 services, PostgreSQL database, and 45 features remain intact but require modifications (see Section 4) — Milestone 2 adds new capabilities on top of them.

What you are adding (four pillars):

a) **Design Patterns** — 7 design patterns from Lab 7 applied at well-defined locations across your codebase (Strategy, Observer, Chain of Responsibility, Builder, Singleton, Factory, Adapter). See Section 3.

b) **Authentication & Authorization** — JWT-based authentication with BCrypt password hashing and role-based access control. The User Service becomes the auth provider; all other services validate tokens independently.

c) **NoSQL Databases** — MongoDB, Redis, Elasticsearch, Neo4j, and Cassandra. Each database serves a specific architectural role.

d) **Caching** — Redis-based caching on all read-heavy endpoints, including your existing Milestone 1 endpoints.

You will also modify your existing M1 codebase (see Section 4) to integrate password hashing, JWT protection, caching, and design pattern retrofits.

Technologies (new in M2): Spring Security, JWT (JJWT), BCrypt, MongoDB, Redis, Elasticsearch, Neo4j, Cassandra, Spring Data for each database.

Spring Boot 4.0.3 on **JDK 25** (Docker base image: `eclipse-temurin:25.0.2_10-jdk`). Jackson dual dependency: Jackson 3.x (`tools.jackson.*`) for Spring Boot, Jackson 2.x (`com.fasterxml.*`) for Hibernate 7.2's JSONB FormatMapper only.

Services (unchanged from M1):

a) **User Service** (port 8081) — Now also the authentication provider

b) **Driver Service** (port 8082) — Now with full-text search via Elasticsearch

c) **Ride Service** (port 8083) — Now with recommendation graph via Neo4j

d) **Location Service** (port 8084) — Now with time-series GPS tracking via Cassandra

e) **Payment Service** (port 8085) — Now with detailed payment analytics via MongoDB

Deliverables: 15 new features (3 per service), JWT authentication on all endpoints, Redis caching on read endpoints, Docker Compose with 6 database containers, continued Git workflow from M1.

Important: This is **not** a microservices milestone. Your services still share a single PostgreSQL database. Inter-service communication comes in Milestone 3. The NoSQL databases are **additive specialized tools**, not replacements for PostgreSQL.

M1 seed data remains valid (additive modifications only): M2 does not alter any M1 PostgreSQL entity schema. The three M1 modifications M2 introduces are all **additive**:

a) **BCrypt password hashing** on the User entity's existing `password` field (see Section 4.1) — the column type is unchanged; only how values are stored changes.

b) **Additive `description` key** in `Driver.vehicleDetails` JSONB (see Section 7.2). Existing Driver rows without this key are tolerated (defaults to empty string).

c) **Additive `surgeFee` key** in `Payment.transactionDetails` JSONB (see Section 4.6). M1's S5-F4 is updated to write this key on new Payments; existing pre-M2 Payments without the key default to `0.15 * amount` (15% surge fallback) inside S5-F10's aggregation.

All existing M1 seed rows, tests, and fixtures continue to work after you apply the M1 modifications in Section 4.

Config format (properties → yml): M1 used `application.properties` per service. In M2 you must migrate each service's configuration to `application.yml`. The auto-grader expects YAML format for M2. Reference fragments are provided in Section 6.5.

2 Git Workflow — Feature Development

The same workflow from Milestone 1 continues. Feature IDs are now **F10–F12** per service:

a) Start from main: `git checkout main && git pull origin main`

b) Create feature branch: `git checkout -b feat/<service>/S{n}-F{m}/<YOUR-STUDENT-ID>`

c) Implement: repository methods → service logic → controller endpoint → test via Postman.

d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"`

e) Push and create a Pull Request on GitHub.

f) At least 1 teammate reviews and approves.

g) **Merge into main using a regular merge commit ("Create a merge commit" on GitHub). Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message, which the auto-grader uses to verify who implemented which feature.

h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

Example branches for M2 features:

- `feat/user/S1-F10/55-8078` — Register User
- `feat/driver/S2-F10/55-8079` — Driver Search
- `feat/ride/S3-F11/55-8080` — Record Riding Pattern
- `feat/location/S4-F12/55-8081` — Location Tracking Timeline
- `feat/payment/S5-F12/55-8082` — Surge-Adjusted Refund

Branch naming convention.

Format: `<type>/<scope>/<descriptor>/<studentID>`

- **type** — one of: `feat`, `fix`, `hotfix`, `refactor`, `docs`, `test`, `chore`, `perf`.
- **scope** — the service or area being touched: `user`, `driver`, `ride`, `location`, `payment` (for per-service work), or `m1`, `cc`, `infra` (for cross-cutting work).
- **descriptor** — the stable ID when the change maps to one (e.g., `S3-F11`, `MOD-3`, `CC-2`), or a short kebab-case otherwise (e.g., `refund-rounding-bug`).
- **studentID** — your numeric student ID, always the last segment; used by the grader.

Examples:

- `feat/ride/S3-F11/55-8080` — new feature (M2)
- `feat/m1/MOD-3/55-8080` — M1 amendment

- `feat/cc/CC-5/55-8080` — cross-cutting requirement
- `fix/payment/refund-amount-rounding/55-8080` — bug fix
- `hotfix/user/token-expiry-leak/55-8080` — urgent fix on an already-merged branch
- `refactor/driver/extract-search-service/55-8080` — internal cleanup, no behavior change

Commit message convention (Conventional Commits).
Format: `<type>(<scope>): <subject> (<studentID>)`

- **type** — same set as branch.
- **scope** — same as branch.
- **subject** — imperative mood ("add", not "added"); no trailing period; keep under 72 characters.
- **studentID** — in parentheses at the end, so every commit is attributable.

Examples:

- `feat(ride): S3-F11 record user-driver riding pattern (55-8080)`
- `feat(m1): MOD-3 apply JWT filter to all M1 endpoints (55-8080)`
- `feat(cc): CC-2 role management endpoint (55-8080)`
- `fix(payment): correct refund amount rounding (55-8080)`
- `refactor(user): extract token validation to helper (55-8080)`
- `test(driver): add integration test for S2-F11 auto-index (55-8080)`
- `docs(cc): clarify CC-5 healthcheck timings (55-8080)`
- `chore(infra): bump postgres image to 17.4 (55-8080)`

Type quick reference.

Type	When to use
`feat`	New feature, amendment, cross-cutting requirement, or design-pattern implementation.
`fix`	Bug fix in existing code (wrong behavior, wrong value, wrong status code).
`hotfix`	Urgent fix on an already-merged branch — use sparingly.
`refactor`	Code change that neither fixes a bug nor adds a feature (renaming, extracting methods).
`docs`	Documentation-only change (README, comments, this spec).
`test`	Tests only (added or fixed).
`chore`	Build, dependency, or tooling change with no runtime impact.
`perf`	Performance improvement without behavior change.

Rules:

- One branch → one PR → one work unit. Don't bundle unrelated changes.
- Always end the branch name and commit subject with your student ID in the exact format shown above — the auto-grader matches on it.
- Prefer the stable IDs (`S<n>-F<m>`, `MOD-<n>`, `CC-<n>`) as the descriptor when the change maps to one. Use a kebab-case slug only for ad-hoc fixes or refactors with no predefined ID.
- Where a design pattern (DP-1..DP-7) is implemented across multiple branches, cite the DP ID in each commit message (e.g., "implements DP-2 Observer").

Important: Any team member who has no commits in the repository, or whose commits cannot be matched to any feature branch, will receive a ***ZERO*** as a result.

Important Note: If any membered missed any of the git checks (like pushed some change directly on the main and not creating a PR for it) or any other thing, it will get reflected on everyone in the team and not that member only. Same goes for any test case or any check in the entire project.

3 Design Patterns

M2 requires you to apply **7 design patterns** from Lab 7 at specific, well-defined locations in your codebase.

3.1 The 7 Required Patterns

#	Pattern	Category	Where Applied
1	Strategy	Behavioral	S5-F12 refund logic
2	Observer	Behavioral	MongoDB event logging across all services
3	Chain of Responsibility	Behavioral	JWT authentication filter chain
4	Builder	Creational	Dashboard and analytics DTOs
5	Singleton	Creational	JwtService / ConfigurationManager
6	Factory	Creational	MongoDB event object creation
7	Adapter	Structural	NoSQL query result → DTO conversion

Distribution: 3 Creational, 1 Structural, 3 Behavioral — balanced across all 3 pattern categories.

3.2 Strategy Pattern — Refund Logic (S5-F12)

Purpose: The S5-F12 refund feature has multiple business rules that must each be encapsulated as a separate strategy (full refund including the surge portion, base-fare-only refund, no-refund when the refund window has expired). Replace if-else chains in the service with a Strategy selector.

Structure:

- `RefundStrategy` interface with `calculateRefund(Payment payment, RefundRequest request)` method returning a `RefundResult` (amount + reason code)
- **3 concrete strategies (required):**
  - `FullRefundWithSurgeStrategy` — returns the full payment amount when `refundSurge=true` and payment is within the 24-hour refund window
  - `BaseFareOnlyRefundStrategy` — returns the payment amount minus the surge fee when `refundSurge=false` and payment is within the 24-hour refund window
  - `NoRefundStrategy` — returns zero and a "refund window expired" reason code when the payment is older than 24 hours from `createdAt`
- A `RefundStrategySelector` (not the service itself) chooses which strategy to instantiate based on `refundSurge` and payment age. The service only calls `selector.select(payment, request).calculateRefund(payment, request)`.

Test scenario:

a) Via reflection, assert that the class `RefundStrategy` exists and is an interface with exactly one abstract method whose name is `calculateRefund`.

b) Via reflection, assert that `FullRefundWithSurgeStrategy`, `BaseFareOnlyRefundStrategy`, and `NoRefundStrategy` all exist and implement `RefundStrategy`.

c) Via reflection, assert that a class named `RefundStrategySelector` (or `RefundStrategyFactory`) exists with a `select`-like method returning `RefundStrategy`.

d) Call S5-F12 with a payment within the 24-hour window and `refundSurge=true` → verify the audit trail in MongoDB contains the strategy name `FullRefundWithSurgeStrategy` (or equivalent identifier stored with the event).

e) Call S5-F12 with a payment within the window and `refundSurge=false` → audit trail records `BaseFareOnlyRefundStrategy`.

f) Call S5-F12 with a payment older than 24 hours → returns 400, audit trail records `NoRefundStrategy`.

g) Inspect the payment-service source: the refund service method must not contain `if (refundSurge)` or equivalent branching.

3.3 Observer Pattern — Event Logging

Purpose: When entity state changes (user registers, ride completes, payment refunded, etc.), events must be logged to MongoDB. Observer decouples logging from business logic.

Structure:

- `EntityObserver` interface with `onEvent(String eventType, Object payload)` method
- Concrete observer: `MongoEventLogger` that writes events to the appropriate MongoDB collection
- Subjects (services) maintain an observer list and call `notifyObservers(eventType, payload)` on state changes. Provide `register(observer)` / `unregister(observer)` methods on each subject.
- Each of the 5 services owns its own per-service `MongoEventLogger` instance; observer registration is not shared across services.

Failure policy Catching Exception for Mongo related features (Soft Dependency): The `MongoEventLogger` must catch any Mongo exception, log it at `WARN` level (`log.warn()`), and **not rethrow**. The upstream Postgres transaction must **not** be rolled back on a Mongo write failure.

Spring vs classical GoF: Spring provides a built-in Observer mechanism via `ApplicationEventPublisher` + `@EventListener`. For M2, **you must implement the classical GoF Observer** (explicit interface + register/notify methods) for learning purposes. No method in any of the 5 services annotated with `@EventListener` may write to MongoDB — all MongoDB event writes must flow through your classical Observer chain. You may, of course, still use Spring for everything else in your services.

Where applied:

- S1 register/login/role-change → observers log `AuthEvent` to `auth_events`
- S2 driver updates → observers log to `driver_events`
- S3 ride state changes → observers log lifecycle events to `ride_events`
- S4 location GPS events → observers log to `location_events`
- S5 refund processed → observers log to `payment_audit_trail`
- **Also applied to your M1 write endpoints** (see Section 4)

Test scenario:

a) Via reflection, assert that `EntityObserver` exists as an interface with an `onEvent(String, Object)` method.

b) Via reflection, assert `MongoEventLogger` class exists and implements `EntityObserver`.

c) Via static analysis (class-file or source scan), assert that no method in the User/Driver/Ride/Location/Payment services annotated with `@EventListener` writes to MongoDB at all — all event-logging to MongoDB must flow through the classical GoF Observer chain.

d) Register a fresh user via `POST /api/auth/register` → verify exactly one `REGISTERED` document exists in `auth_events` collection with matching `userId` and recent `timestamp`.

e) Log in as that user → verify a `LOGGED_IN` document is added.

f) Trigger an M1 write (e.g., `PUT /api/users/{id}/preferences` from S1-F2) → verify the corresponding event document is written to MongoDB (confirms the retrofit path).

g) Unregister all observers from a service in a unit test and repeat the write → no MongoDB event should appear (proves the logging path goes through the observer chain, not a direct Mongo call).

3.4 Chain of Responsibility — JWT Filter Chain

Purpose: JWT authentication has multiple sequential steps (extract token, verify signature, load user, check role). Each step is a separate handler in a chain.

Structure:

- `AuthHandler` abstract class (or interface) with `handle(AuthContext ctx)` and `setNext(AuthHandler next)` methods, where `AuthContext` carries the HTTP request, extracted token, authenticated user, and required role
- Concrete handlers (in order): `TokenExtractionHandler` (401 if missing), `SignatureValidationHandler` (401 if invalid/expired), `UserLoaderHandler` (401 if user not found in PG), `RoleAuthorizationHandler` (403 if insufficient role for the endpoint)
- Each handler does its job and passes to the next; returns an error result immediately if any check fails

Spring Security integration: Spring Security's `SecurityFilterChain` is itself a chain of responsibility at the framework level. You **must not** replace Spring Security's filter chain. Instead, build your custom `AuthHandler` chain **inside** your `JwtAuthenticationFilter.doFilterInternal()` method:

- Spring Security's filter chain invokes your `JwtAuthenticationFilter` once per request
- Inside `doFilterInternal`, construct (or inject) the head of your custom `AuthHandler` chain and call `head.handle(new AuthContext(request))`
- If any handler in your chain fails, write the appropriate status code (401 or 403) to the response and short-circuit by **not** calling `filterChain.doFilter(...)`
- If all handlers succeed, populate the Spring Security context (`SecurityContextHolder.getContext().setAuthentication(...)`) and call `filterChain.doFilter(...)`

Test scenario:

a) Via reflection, assert `AuthHandler` class/interface exists with `setNext(AuthHandler)` and `handle(...)` methods.

b) Via reflection, assert at least 3 concrete subclasses of `AuthHandler` exist (e.g., `TokenExtractionHandler`, `SignatureValidationHandler`, `UserLoaderHandler`, and optionally `RoleAuthorizationHandler`).

c) Call a protected M1 endpoint (e.g., `GET /api/users/1`) with `Authorization` header absent → returns 401 (TokenExtractionHandler failed).

d) Call with `Authorization: Bearer invalid.token.here` (invalid signature) → returns 401 (SignatureValidationHandler failed).

e) Delete the user from PG, then call with a valid token issued for that user → returns 401 (UserLoaderHandler failed).

f) Call `PUT /api/users/1/role` with a RIDER token → returns 403 (RoleAuthorizationHandler failed).

g) Call the same endpoint with an ADMIN token → succeeds (chain passes through).

h) Inspect `JwtAuthenticationFilter.doFilterInternal()` source: the grader parses it and confirms that the method body invokes the head of the `AuthHandler` chain rather than duplicating the extraction/validation/authorization logic inline.

3.5 Builder Pattern — Dashboard DTOs

Purpose: Dashboard and analytics DTOs have 5–10+ fields. Builder improves readability and allows optional fields.

Structure:

- Static inner `Builder` class on each dashboard DTO (or external Builder class)
- `builder()` static method to start construction
- Fluent setters returning `this`
- `build()` method returning the DTO instance

Builder with Java records: The project convention is "Java records for DTOs." Records do not naturally support Builder (the canonical constructor takes all fields positionally). Two options, both accepted by the grader:

- **Convert to class:** Turn the record into a class with a static inner `Builder`.
- **External builder on the record:** Keep the record and write an external `<DtoName>Builder` class whose `build()` method constructs the record via its canonical constructor.

Where applied in M2: S2-F12 `DriverDashboardDTO`, S3-F10 `RideAnalyticsDashboardDTO`, S4-F10 `LocationAnalyticsDTO`, S5-F10 `VehicleTypeRevenueDTO`.

Uber M1 retrofit scope: Apply Builder to M1 DTO-returning features whose DTOs have 5+ fields. These DTOs for Uber specifically are:

- **S1-F3, S1-F6, S1-F8, S1-F9:** Builder required (all return DTOs with 5+ fields)
- **S2-F3, S2-F6, S2-F9:** Builder required
- **S2-F8 (Verify Driver Document): No Builder retrofit** — this feature returns an entity, not a DTO
- **S3-F3, S3-F6, S3-F9:** Builder required
- **S3-F8 (Add Stops to Existing Ride): No Builder retrofit** — returns an entity
- **S4-F3, S4-F6, S4-F8, S4-F9:** Builder required
- **S5-F3, S5-F6, S5-F8, S5-F9:** Builder required

Test scenario:

a) Via reflection, iterate every DTO class returned by an M2 dashboard feature: assert each has an accessible static `builder()` method, that chaining setters returns the Builder, and that `build()` returns the correct DTO type.

b) Via reflection, iterate the in-scope Uber M1 DTOs (`UserRideSummaryDTO`, `TopRiderDTO`, `DriverEarningsDTO`, etc.): each must have a Builder.

c) In an integration test, call S2-F12 `GET /api/drivers/{id}/dashboard` → verify the response is correctly populated (proves Builder is used to construct the DTO before serialization).

d) Call M1 S1-F3 `GET /api/users/{id}/ride-summary` → verify it still returns the expected fields (proves the retrofit did not break M1 behavior).

e) Compile-time or static analysis check: confirm S2-F8 and S3-F8 do **not** use Builder (they return entities, not DTOs).

3.6 Singleton Pattern — JwtConfigurationManager

Purpose: A class holding shared, immutable JWT configuration (secret key, expiration, algorithm) should have exactly one instance.

Structure:

- Private constructor
- Static `getInstance()` method with thread-safe initialization (double-checked locking or eager init)
- Single private static field holds the instance

Loading config in a non-Spring class: Since `JwtConfigurationManager` is not a Spring bean, it cannot use `@Value` or `@ConfigurationProperties`. Use one of the following accepted approaches:

- **(Recommended) Environment variables with fallback defaults:** Inside the private constructor, read `System.getenv("JWT_SECRET")`, `System.getenv("JWT_EXPIRATION_MS")`, etc., with sensible fallback defaults for local dev. Docker Compose already sets the required env vars.
- **Singleton-bridge pattern:** A Spring `@Configuration` bean reads `application.yml` via `@Value`, then pushes the values into the singleton via a static setter (e.g., `JwtConfigurationManager.initConfig(secret, expirationMs)`) during `@PostConstruct`. The singleton then serves those values through its instance methods.

Both approaches are accepted by the grader as long as the class itself is not a Spring bean and `getInstance()` returns a consistent singleton reference.

Spring vs classical GoF (important): Spring's `@Component`/`@Service` beans are singletons by default, but they are managed by the Spring container, not by the GoF pattern. For M2 you must have **exactly one** class implemented as a classical GoF Singleton:

- **Implement `JwtConfigurationManager` as a classical Singleton** (private constructor + `getInstance()`). This class is not a Spring bean. Do not annotate it with `@Component`.
- `JwtService` **remains a Spring** `@Service`. It obtains JWT config via `JwtConfigurationManager.getInstance()` rather than via `@Autowired`.
- All other infrastructure classes (controllers, services, repositories) remain Spring-managed.

Test scenario:

a) Via reflection, assert `JwtConfigurationManager` exists and has exactly one constructor declared with `private` access.

b) Via reflection, assert `getInstance()` is a `public static` method returning `JwtConfigurationManager`.

c) Call `getInstance()` twice in the test and assert `ref1 == ref2` (reference equality, not just `.equals`).

d) Spawn 10 parallel threads each calling `getInstance()` → all must return the same reference (thread-safety under contention).

e) Via reflection, assert the class is NOT annotated with `@Component`, `@Service`, `@Configuration`, or any other Spring stereotype.

f) In an integration test, verify that `JwtService` (a Spring bean) correctly reads the configured secret via `JwtConfigurationManager.getInstance()` — issue and validate a token and confirm the flow works.

3.7 Factory Pattern — Event Creation

Purpose: MongoDB events have different types across services (`AuthEvent`, `DriverEvent`, `RideEvent`, `LocationEvent`, `PaymentAuditEvent`). A `EventFactory` creates the right concrete event type based on input.

Structure:

- A common `MongoEvent` interface that all 5 concrete event classes implement, with methods `getId()`, `getTimestamp()`, `getAction()`, `getDetails()`. (See Section 7.1 for the common interface definition.)
- `EventFactory` class with `createEvent(EventType type, Map<String, Object> params)` method
- The factory dispatches on `type` (an enum with values `AUTH, DRIVER, RIDE, LOCATION, PAYMENT_AUDIT`) and returns the appropriate concrete class, typed as `MongoEvent`

How Observer and Factory compose: On an M1 write completing, the service calls `notifyObservers(eventType, payload)`. The `MongoEventLogger` observer receives the call, asks the `EventFactory` for the right event subtype, and persists the event to MongoDB via Spring Data.

Test scenario:

a) Via reflection, assert `MongoEvent` interface exists with methods `getId`, `getTimestamp`, `getAction`, `getDetails`.

b) Via reflection, assert all 5 event classes (`AuthEvent`, `DriverEvent`, `RideEvent`, `LocationEvent`, `PaymentAuditEvent`) implement `MongoEvent`.

c) Via reflection, assert `EventFactory` exists with a `createEvent(EventType, Map<String, Object>)` method.

d) In a unit test, call `EventFactory.createEvent(AUTH, params)` → assert the returned object is assignable to `AuthEvent` and its fields match the params.

e) Repeat for each of the 5 `EventType` values.

f) Call `EventFactory.createEvent(PAYMENT_AUDIT, ...)` → the returned `PaymentAuditEvent` must expose `method` and `amount` fields (service-specific fields added on top of the common interface).

g) Integration test: register a user via `POST /api/auth/register` → inspect the resulting MongoDB document in `auth_events`. Its type and fields must match what `EventFactory.createEvent(AUTH, ...)` would produce (proves services go through the factory).

h) Source-scan check: verify no service class contains `new AuthEvent(...)`, `new DriverEvent(...)`, etc. — all event construction must go through the factory.

3.8 Adapter Pattern — NoSQL Result → DTO Conversion

Purpose: Results from different NoSQL databases have different raw shapes (MongoDB `Document`, Elasticsearch `SearchHit`, Neo4j `Record`, Cassandra `Row`). Adapters convert each raw source to the specific DTO the service returns.

Structure:

- Each adapter has a single `adapt(source)` → `targetDto` method. There is **no** universal "Entity-Dto" base type — each adapter converts to its specific domain DTO.
- One adapter per NoSQL source used in the service:
  - S1: `MongoDocumentAdapter` (for `AuthEvent` documents)
  - S2: `MongoDocumentAdapter`, `ElasticsearchHitAdapter`
  - S3: `MongoDocumentAdapter`, `Neo4jRecordAdapter`
  - S4: `MongoDocumentAdapter`, `CassandraRowAdapter`
  - S5: `MongoDocumentAdapter`

M1 retrofit scope (conditional): In M1, some features that return DTOs from native SQL used `Object[]` row projections while others used JPQL constructor expressions or `@Query` DTO projections. The Adapter retrofit is **conditional**:

- **If** your M1 feature uses `Object[]` (S1-F3 in the M1 spec explicitly mandates this) → wrap the mapping in an `ObjectArrayDtoAdapter` class rather than doing inline mapping in the service.
- **If** your M1 feature uses JPQL constructor expressions or DTO projections → no Adapter needed; the existing code satisfies the pattern by construction.

Test scenario:

a) Via reflection, assert that for each NoSQL source used by each service, a corresponding adapter class exists: `MongoDocumentAdapter` in all services; `ElasticsearchHitAdapter` in driver-service; `Neo4jRecordAdapter` in ride-service; `CassandraRowAdapter` in location-service.

b) Via reflection, assert each adapter has an `adapt(...)` method whose return type matches the service's domain DTO.

c) In a unit test, pass a mock MongoDB `Document` to `MongoDocumentAdapter.adapt(...)` → verify the returned DTO has fields populated from the document's keys.

d) In a unit test for driver-service: pass an Elasticsearch `SearchHit` to `ElasticsearchHitAdapter.adapt(...)` → verify the returned driver DTO.

e) For S1-F3 (the only M1 feature that explicitly mandates `Object[]`): via reflection or source scan, verify an `ObjectArrayDtoAdapter` (or similar) class exists and is used to convert the native SQL `Object[]` rows into `UserRideSummaryDTO`. Invoke S1-F3 in integration and verify the output is correct.

f) For any other in-scope M1 F3/F6/F9 feature that the team chose to implement with `Object[]`: check the corresponding adapter exists. Features implemented via JPQL constructor expressions or DTO projections are **exempt** from this check.

3.9 Grading Summary

Each pattern contributes roughly equally to the design patterns portion of the M2 grade. The grader performs:

- **Static analysis** — reflective inspection of class structure (interfaces, methods, constructors)
- **Behavioral verification** — integration tests that exercise the patterns indirectly through M2 features

4 M1 Modifications Required

Before you can build M2 features, you must **modify your existing M1 codebase** in specific ways. These modifications are prerequisites — M2 features and cross-cutting requirements depend on them.

4.1 User Entity — Password Hashing

Your M1 User entity already has a `password` field. In M1 it was stored as plaintext (or loosely). In M2:

- All new user registrations must hash passwords with **BCrypt** before saving
- Existing M1 test users/seeds should be re-created with hashed passwords (or hash them on first login)
- The `password` field must **never** be returned in any API response (including existing M1 `GET /api/users/{id}` endpoint — filter it out in the DTO)

Test scenario:

a) Register a user via `POST /api/auth/register` with password `"securePassword123"`.

b) Query the `users` table directly and assert the stored `password` column starts with `"$2a$"`, `"$2b$"`, or `"$2y$"` (BCrypt hash prefix) and its length is 60 characters.

c) Assert the stored password is **not** equal to `"securePassword123"`.

d) `POST /api/auth/login` with the same credentials → succeeds (BCrypt verify works).

e) Call `GET /api/users/{id}` (M1 endpoint) with a valid token → inspect the JSON response body: the `password` field must be absent or explicitly `null`. Same for any other endpoint that returns a User (e.g., S1-F1 search, S1-F3 summary).

f) Re-seed the database using the seed mechanism → assert every seeded user's `password` column is a BCrypt hash, not plaintext.

4.2 User Entity — Role Values (Additive)

Your M1 User entity already has a `role` ENUM with values `RIDER` and `ADMIN`. **Do not remove or rename existing M1 role values.** M2 modifies this only additively:

- **Keep `RIDER` and `ADMIN`** as they are
- **Default role on registration:** `RIDER` (Uber's M1 default). `ADMIN` is never assigned at registration.
- **ADMIN assignment:** Only via the role management endpoint (`PUT /api/users/{id}/role`) which itself requires ADMIN. Seed at least one ADMIN user via your M1 seed mechanism so role management can be tested.

JWT role claim: The token carries the user's role. Authorization logic:

- **USER-level endpoints:** Accept any authenticated user (`RIDER` or `ADMIN`) — this is the default for M2 features.
- **ADMIN-only endpoints:** Require `role == ADMIN`. Only the role management endpoint uses this.

Test scenario:

a) Query PostgreSQL metadata (`information_schema.columns` or the Postgres enum `pg_enum`) to confirm the `role` ENUM type contains both `RIDER` and `ADMIN` values.

b) Register a fresh user via `POST /api/auth/register` with valid data → query the `users` table and assert the new row's `role` column is `RIDER`.

c) Decode the returned JWT token → assert the `role` claim is `RIDER`.

d) Log in as a seeded ADMIN user → decode token and assert `role` claim is `ADMIN`.

e) Attempt to register with a request body containing `"role": "ADMIN"` (a bad-faith attempt) → the server must ignore that field and still create a `RIDER`.

f) With the RIDER token, call `PUT /api/users/{id}/role` → 403 Forbidden.

g) With the ADMIN token, call `PUT /api/users/{id}/role` with body `{"role":"ADMIN"}` → 200, the target user's role changes.

h) With the RIDER token, call any regular M2 feature (e.g., S2-F10 search) → succeeds (USER-level endpoints accept both RIDER and ADMIN).

4.3 Existing M1 Endpoints — JWT Authentication

All existing M1 endpoints (45 feature endpoints + CRUD endpoints on all entities) must now require a valid JWT token, except:

- `POST /api/auth/register` (new in M2)
- `POST /api/auth/login` (new in M2)
- Health check endpoints

Test scenario:

a) Call `POST /api/auth/register` (public) → succeeds without any `Authorization` header.

b) Call `POST /api/auth/login` (public) → succeeds without a token.

c) Call any M1 feature endpoint (e.g., `GET /api/drivers/search`, M1 S2-F1) without token → returns 401.

d) Call the same endpoint with `Authorization: Bearer invalid` → returns 401.

e) Call the same endpoint with a valid token → succeeds with 200 and the expected M1 response shape.

f) Call every CRUD endpoint (`GET /api/users`, `GET /api/users/{id}`, `POST /api/users`, `PUT /api/users/{id}`, `DELETE /api/users/{id}`, and the equivalents on all 10 Uber entities) without token → each returns 401. With a valid token → each returns the expected M1 status code.

g) Health check endpoints (e.g., `GET /actuator/health`) return 200 without a token.

h) Confirm by grepping the test results that all 45 M1 feature tests + CRUD tests pass under M2 auth.

4.4 Existing M1 Read Endpoints — Redis Caching (Explicit Enumeration)

All M1 read-heavy endpoints must be cached in Redis. Below is the complete enumeration — nothing is left open.

4.4.1 M1 Feature GET Endpoints That Must Be Cached (Uber: 27 total)

The canonical M1 pattern is "F2, F4, F7 are writes, the other 6 are reads," but **Uber deviates at three specific features** that are writes despite their F-number:

Service	Feature	Verb	Why it is a write
S2 (Driver)	S2-F8 Verify Driver Document	PUT	Modifies `DriverDocument.verified` and writes verificat
S3 (Ride)	S3-F8 Add Stops to Existing Ride	POST	Creates new `RideStop` rows
S5 (Payment)	S5-F5 Apply Coupon to Payment	POST	Creates `PaymentCoupon` join row and mutates Payment

These are **excluded** from the cached-reads list and **included** in the invalidating-writes list (Â§4.4.4).
The explicit cached-read enumeration for Uber is:

Service	Cached GET feature IDs	C
S1 (User)	F1, F3, F5, F6, F8, F9	6
S2 (Driver)	F1, F3, F5, F6, F9 (F8 is a write)	5
S3 (Ride)	F1, F3, F5, F6, F9 (F8 is a write; F3 is a POST estimate—see note)	5
S4 (Location)	F1, F3, F5, F6, F8, F9	6
S5 (Payment)	F1, F3, F6, F8, F9 (F5 is a write)	5
Total cached feature endpoints (Uber)		**2**

Note on S3-F3 (Get Fare Estimate): In the M1 spec S3-F3 is a `POST /api/rides/fare-estimate` that computes a fare estimate without persisting. It is semantically read-only despite being POST. Cache it by request-body hash (TTL 5 min). If you did not implement S3-F3 as a POST, follow the M1 spec verb for your service.

TTLs by feature type: F1 = 5 min (search), F3 = 10 min (DTO), F5 = 5 min (JSONB query), F6 = 10 min (report), F8 = 15 min (relationship DTO), F9 = 10 min (combined).

4.4.2 CRUD Baseline Endpoints That Must Be Cached

For every entity, **only** the get-by-ID endpoint is cached. List endpoints are **not** cached — they hit PostgreSQL on every request.

Endpoint	Cached?	TTL
`GET /api/<entity>` (list all)	No	—
`GET /api/<entity>/{id}` (get by ID)	Yes	15 min

Uber entities (10): user, saved-address, driver, driver-document, ride, ride-stop, location, payment, coupon, payment-coupon. **10 GET-by-ID endpoints must be cached.**

4.4.3 Total

27 M1 feature endpoints + 10 M1 CRUD GET-by-ID endpoints = 37 M1 endpoints must be cached for Uber. Plus the M2 feature GET endpoints.

4.4.4 Endpoints That Must Invalidate Caches (Writes)

Writes invalidate the detail cache of the specifically affected entity ID, plus feature-result caches whose output includes that entity. Since list endpoints are not cached, there is no list cache to invalidate.

Uber M1 feature writes (18 total):

- S1 writes: F2, F4, F7 (3)
- S2 writes: F2, F4, F7, **F8 (Verify Driver Document)** (4)
- S3 writes: F2, F4, F7, **F8 (Add Stops to Existing Ride)** (4)
- S4 writes: F2, F4, F7 (3)
- S5 writes: F2, F4, **F5 (Apply Coupon to Payment)**, F7 (4)

M1 CRUD writes (30): `POST`, `PUT`, `DELETE` on each of the 10 Uber entities.

- `POST /api/<entity>` (create) — nothing to invalidate (new entity has no cached detail yet)
- `PUT /api/<entity>/{id}` (update) — invalidate detail cache for that specific ID + any feature caches referencing that entity
- `DELETE /api/<entity>/{id}` — invalidate detail cache for that specific ID + any feature caches referencing that entity

Total Uber M1 endpoints that invalidate: 18 feature writes + 30 CRUD writes = 48.

M2 writes that also invalidate M2 feature caches: In addition to the M1 writes above, the following M2 operations must invalidate feature-result caches so that read-side views see the change before the TTL expires:

- CC-2 Role management (`PUT /api/users/{id}/role`) — must invalidate `user-service::user::{id}` (detail) and all keys matching `user-service::S1-F12::*` for that `userId` (so the ROLE_CHANGED event appears in the activity feed immediately).
- Any M1 write that creates/updates a `Ride` referencing a specific `driverId` — must invalidate `driver-service::S2-F12::{driverId}` so the Driver Performance Dashboard reflects the new activity.
- Any M1 write that creates/updates a `Ride` — must invalidate `ride-service::S3-F10::*` (the Ride Analytics dashboard aggregates over all rides).
- Any M1 write that creates/updates a `Location` — must invalidate `location-service::S4-F10::*`.
- Any M1 write that creates/updates a `Payment` (including S5-F4 Process Payment, S5-F2 Refund, M2 S5-F12 Refund) — must invalidate `payment-service::S5-F10::*` and `payment-service::S5-F11::*`.

NoSQL-writer → cached-reader invalidation (required): Several M2 features write to a NoSQL store as their primary action (no PG row mutates), but downstream M2 features cache reads of that same NoSQL store. The PG-write rules above do **not** cover these paths, so add these explicitly:

- **S4-F11** (`POST /api/locations/{driverId}/tracking`) writes a new tracking row to Cassandra and emits a `TRACKING_RECORDED` event to MongoDB `location_events` — must invalidate `location-service::S4-F12::{driverId}` (the cached tracking timeline for that driver, 5 min TTL) so S4-F12 reflects the new tracking point immediately. Must also invalidate `location-service::S4-F10::*` (the location analytics dashboard may aggregate over tracking-event counts).
- **S3-F11** (`POST /api/rides/{rideId}/record-interaction`) writes a new `RODE_WITH` relationship (or increments an existing one) in Neo4j — must invalidate `ride-service::S3-F12::*` (the cached recommendation results for all users, 5 min TTL) because the recommendation graph changed. Use wildcard deletion rather than narrow targeting: a new edge between user U and driver D can alter recommendations for every user who shares a driver with U, not just U.

- **S2-F11** (`POST /api/drivers/{id}/index`) **and** every Driver CRUD write that triggers the auto-indexing retrofit (see Â§10 S2-F11 step f) — must invalidate `driver-service::S2-F10::*` (the cached full-text search results, 5 min TTL). Driver name / description / status changes alter ES relevance ranking, so stale cache can return deleted or renamed drivers.
- **S5-F10 / S5-F11 analytics writes** — whenever any Observer writes a **data-mutating** event to `payment_audit_trail` (specifically `CREATED`, `COMPLETED`, `FAILED`, `REFUNDED`, `REFUND_DENIED`, `COUPON_APPLIED`), the payment-service analytics caches `payment-service::S5-F10::*` and `payment-service::S5-F11::*` must be invalidated. This covers both the normal-processing path (S5-F4, S5-F2, S5-F5, S5-F12 success branch) and the strategy-denial path (S5-F12 NoRefund-Strategy branch logs `REFUND_DENIED` without any PG mutation — the PG-write rules above miss this case, so the analytics caches would otherwise serve stale data until TTL expiry). Similarly, any Observer write of a data-mutating action to `driver_events`, `ride_events`, or `location_events` invalidates the corresponding `S2-F12` / `S3-F10` / `S4-F10` analytics keys.
- **Pure observability actions do NOT invalidate caches** — specifically `ANALYTICS_VIEWED` and `DASHBOARD_VIEWED`. These actions are written by the four dashboard endpoints (S2-F12, S3-F10, S4-F10, S5-F10) on *every* invocation (including cache hits; see Â§10) purely for audit-trail / usage-telemetry purposes. They do not change what the analytics queries return. Triggering wildcard invalidation on them would create a self-defeating cycle: every cached dashboard call would log an observability event, which would wildcard-invalidate its own key, guaranteeing that the next call is always a miss and the cache never serves a hit. Exclude both `ANALYTICS_VIEWED` and `DASHBOARD_VIEWED` from the invalidation trigger in the Observer (match on the `action` field before invalidating).

These M2 invalidation rules compose with the M1 rules above; implement both.

4.4.5 Cache Key Convention

- Entity detail: `<service>::<entity>::<id>` (e.g., `user-service::user::42`)
- Feature result: `<service>::<featureId>::<param-hash>` (e.g., `user-service::S1-F3::42`)

4.4.6 Cache Invalidation Strategy (Wildcard Deletion)

Computing the exact set of feature-result caches affected by an entity change is infeasible without cache tags or reverse indexes. Use **wildcard key deletion** instead:

- When entity `X` with id `N` changes, delete the key `<service>::X::N` (entity detail).
- Also delete all feature-result keys matching the pattern `<service>::S{n}-F{m}::*` for every feature `S{n}-F{m}` whose output *may* include entity `X` (conservatively identified at design time).
- This over-invalidates (clears some entries that did not actually change). That is acceptable for M2 — correctness beats cache-hit optimality.

Use the Redis `SCAN` + `DEL` combination or `KEYS-UNLINK` (for small caches) to implement wildcard deletion.

Test scenario (entire Â§4.4 caching retrofit):

a) For each of the 27 cached M1 feature GET endpoints listed in Â§4.4.1: call it twice with identical parameters and a valid token. Inspect Redis between calls (e.g., via `redis-cli KEYS 'user-service::*'`) and assert a matching key exists. Measure the second call's latency — it must be less than the first (cache hit).

b) For each of the 10 CRUD `GET /api/<entity>/{id}` endpoints: call twice and verify a key `<service>::<entity>::<id>` exists in Redis after the first call.

c) Call `GET /api/drivers` (list) twice → assert **no** cache key was created for the list (list endpoints are NOT cached).

d) Fetch driver ID 5 (cache it), then `PUT /api/drivers/5` with an update → verify the key `driver-service::driver::5` is removed from Redis. Next GET recomputes from PG.

e) Fetch the S1-F3 DTO for user 10 (cache it), then update user 10 via `PUT /api/users/10/preferences` (S1-F2) → verify every key matching `user-service::S1-F3::*` is removed (wildcard invalidation).

f) Delete driver 7 via `DELETE /api/drivers/7` → the cached `driver-service::driver::7` must disappear.

g) Disable Redis (e.g., stop the container) and retry a cached endpoint → it still returns correct data from PG (soft-dependency graceful degradation).

h) Verify TTLs: fetch an S1-F1 search result (5 min TTL), wait 5m+, fetch again → cache was auto-evicted and the call recomputes.

4.5 Design Pattern Retrofits to M1

The 7 patterns from Section 3 apply to M2 code **and** to M1 retrofits as follows:

Pattern	Applies to M1?	Which M1 Uber features	Notes
Strategy	No	—	New M2 code only (S5-F12 refund) — do NOT force onto any M1 method
Observer	Yes (universal)	All M1 write endpoints: F2, F4, F7 per service + S2-F8, S3-F8, S5-F5 (Uber-specific writes) + all CRUD writes	State change → `notifyObservers` → MongoDB event log
Chain of Responsibility	Yes (universal)	All M1 endpoints via the JWT filter chain	Inside `JwtAuthenticationFilter` (see Section 3.4)
Builder	Yes	S1-F3, S1-F6, S1-F8, S1-F9; S2-F3, S2-F6, S2-F9; S3-F3, S3-F6, S3-F9; S4-F3, S4-F6, S4-F8, S4-F9; S5-F3, S5-F6, S5-F8, S5-F9 (DTO-returning features with 5+ fields)	**Excludes** S2-F8 and S3-F8 for Uber — they return entities, not DTOs
Singleton	No	—	New M2 code only (JwtConfigurationManager)
Factory	Yes (indirect)	M1 write endpoints trigger Observer; Observer calls `EventFactory`	EventFactory creates the right MongoEvent subtype
Adapter	Conditional	Any M1 feature that uses `Object[]` native SQL projection (S1-F3 mandates `Object[]`; other F3/F6/F9 only if you chose `Object[]` in M1)	If you used JPQL constructor expressions or DTO projections, no Adapter retrofit is required

Important: Strategy Pattern is used **only** in M2 code (S5-F12 refund). Do not force it onto any M1 method.

Composition workflow (Observer + Factory + Adapter):

a) A write endpoint (e.g., M1 S1-F2 Update User Preferences) completes its PG update.

b) The service calls `notifyObservers(''USER_UPDATED'', userPayload)` before returning. The first argument is the **action string** (what happened); it is not the `EventType` passed to the factory.

c) `MongoEventLogger` observer receives the call. Each of the 5 services instantiates its own `MongoEventLogger` bound to a **fixed** `EventType` at construction time (user-service binds `AUTH`, driver-service binds `DRIVER`, ride-service binds `RIDE`, location-service binds `LOCATION`, payment-service binds `PAYMENT_AUDIT`). The logger does **not** infer `EventType` from the action string.

d) The logger builds the factory input: copy the action string into `params.put("action", eventType)` plus the rest of the payload, then calls `EventFactory.createEvent(boundEventType, params)`.

e) The factory returns the matching concrete `MongoEvent` subtype (e.g., `AuthEvent`) typed through the common interface.

f) The logger persists the event to MongoDB via Spring Data's repository.

Adapter sits separately on **read paths** (converting NoSQL query results or M1 `Object[]` results to DTOs) — it does not participate in the write-side composition above.

Payment Service payment_audit_trail population (important): The `payment_audit_trail` collection powers M2 S5-F11 (`GET /api/payments/analytics/methods`). Without writes to this collection, S5-F11 returns empty results. Therefore:

- **M1 S5-F4** (Process Payment) must write two events to `payment_audit_trail`: a `CREATED` event when the Payment row is inserted, and a `COMPLETED` event when the payment status transitions to `COMPLETED`.
- **M1 S5-F2** (Process Refund — the original M1 refund) must write a `REFUNDED` event.

- **M2 S5-F12** (Process Ride Refund with Surge Handling) must also write a `REFUNDED` event with richer details (strategy name, surge inclusion).

These are observer-driven writes — use the Observer retrofit (row 2 of the table above) rather than inline Mongo calls in the service methods.

Additional M1 behavior retrofits:

- **S5-F4 gateway-failure simulation (query param):** M1's `POST /api/payments/ride/{rideId}` must accept an optional `?simulateFailure=true` query parameter. When set, S5-F4 short-circuits to `Payment.status = FAILED` in PostgreSQL and writes a `FAILED` event to `payment_audit_trail` (via Observer) instead of the normal CREATED→COMPLETED flow. When absent or `false`, S5-F4 behaves identically to its M1 spec. This affordance exists solely so the S5-F11 test scenario can produce failed-payment data; it is not a production flow. See the `FAILED` row of Â§7.1.6 PaymentAuditEvent for the full write-conditions table.
- **Driver CRUD auto-index to Elasticsearch:** M1's Driver CRUD controller must auto-sync to the M2 Elasticsearch index on every write. `POST /api/drivers` and `PUT /api/drivers/{id}` must **re-index** the driver's search document (as if S2-F11 had been called explicitly); `DELETE /api/drivers/{id}` must **remove** the document from the index. See Â§10 S2-F11 step f for the exact expected behavior. Implement via a `@PostPersist`/`@PostUpdate`/`@PostRemove` JPA entity listener or a service-level hook; do **not** inline the ES call into every CRUD controller method.

Test scenario (design pattern retrofits):

a) Call M1 S1-F2 `PUT /api/users/{id}/preferences` → verify an event document appears in `auth_events` (Observer retrofit fired on the write).

b) Call M1 S2-F8 (Verify Driver Document) → event in `driver_events`.

c) Call M1 S3-F8 (Add Stops to Existing Ride) → event in `ride_events`.

d) Call M1 S5-F5 (Apply Coupon to Payment) → event in `payment_audit_trail`.

e) Call M1 S5-F4 (Process Payment) → `CREATED` and `COMPLETED` events appear in `payment_audit_trail`.

f) Call M1 S5-F2 (Process Refund) → `REFUNDED` event appears.

g) Call a CRUD write (e.g., `POST /api/drivers`) → event in `driver_events`.

h) Call M1 S1-F3 `GET /api/users/{id}/ride-summary` → response is well-formed; via source scan, confirm the DTO is constructed via its Builder (not via `new UserRideSummaryDTO(...)` directly inside the service).

i) Call M1 S1-F3 and confirm that the Object[] result from the native SQL query is converted to `UserRideSummaryDTO` via an Adapter class (not inline mapping in the service).

j) Verify that Strategy is NOT used anywhere in M1 service classes (grep for `RefundStrategy`, `Strategy` outside the payment service's S5-F12 package).

4.6 Payment.transactionDetails — Additive `surgeFee` Key

M2 S5-F10 (Fare Revenue by Vehicle Type with Surge Breakdown) requires the Payment's surge-fee portion to be separable from the base-fare revenue. M1's `Payment.transactionDetails` JSONB schema is `{gatewayResponse, cardLastFour, receiptUrl, failureReason}` — it does **not** include a `surgeFee` key.

M2 additively extends this JSONB with a new key:

- `surgeFee` — numeric amount (currency units) representing the surge-pricing portion of the total payment.

Required M1 code change: Update M1's S5-F4 (Process Payment) to compute the surge fee when creating the Payment and write it into `transactionDetails.surgeFee`. Derive the surge fee from the Ride's `metadata.surgeMultiplier` if present (`surgeFee = baseFare * (surgeMultiplier - 1)`), or as a flat 15% of the total amount if no surge multiplier is set.

Fallback for pre-M2 rows: Existing M1 Payment rows created before this modification will not have a `surgeFee` key. Any feature reading the key (e.g., S5-F10) must treat a missing or null `surgeFee` as `0.15 * payments.amount` (15% of total). This keeps pre-M2 data usable without a DB backfill migration.

4.7 Summary Checklist

Before starting M2 features, ensure your M1 codebase:

- ☐ Password hashing (BCrypt) applied to registration and seed data
- ☐ `ADMIN` role present; `RIDER` preserved as default
- ☐ At least one `ADMIN` user seeded
- ☐ All endpoints (except register/login/health) require JWT
- ☐ 27 M1 feature GET endpoints + 10 CRUD GET-by-ID endpoints cached (37 total — see Section 4.4)
- ☐ M1 write endpoints invalidate the correct detail/feature caches
- ☐ M1 write endpoints notify observers (event logging)
- ☐ M1 complex DTOs (F3/F6/F8/F9) use Builder pattern
- ☐ M1 `Object[]` native SQL results (F3/F6/F9) use Adapter pattern
- ☐ M1 Driver rows carry a `vehicleDetails.description` key (default empty) — see Section 7.2
- ☐ M1 S5-F4 writes `surgeFee` to `Payment.transactionDetails` JSONB — see Section 4.6

5 Authentication & Authorization

5.1 Overview

The User Service (S1) becomes the authentication provider. It exposes register and login endpoints that return JWT tokens. All other services validate tokens independently using a shared secret key — no inter-service calls are needed for authentication.

5.2 JWT Token

- **Algorithm:** HMAC-SHA256
- **Secret:** A shared key configured in each service's `application.yml` (e.g., `jwt.secret`). The shared secret must be at least **32 bytes (256 bits)** of entropy when decoded — JJWT's HS256 signer enforces this minimum and throws `WeakKeyException` at construction time on shorter keys. Use `Keys.secretKeyFor(SignatureAlgorithm.HS256)` to generate a compliant key, or configure any 44-character Base64-encoded string of random bytes (44 Base64 chars decode to 32 bytes). Short readable strings like `"mySecret123"` will fail at the first token-issue attempt.

- **Payload:** The token contains the user's email (as the `sub` subject claim), the user's numeric `User.id` (as a custom `uid` claim), the user's role (as a `role` claim), issued-at (`iat`), and expiration (`exp`). Ownership checks in Â§10 (S1-F12, S3-F12) compare the `uid` claim directly against the path/query `userId` — no additional PG lookup is required on the hot path.
- **Header format:** `Authorization: Bearer <token>`
- **Expiration:** Configurable (e.g., 24 hours)

5.3 Roles (Additive — Preserve M1 Values)

The M1 User entity already has a `role` ENUM with values `RIDER` and `ADMIN`. M2 preserves these additively — do not rename or remove existing M1 role values.

Role	Permissions
`RIDER` (M1 default)	Access all standard endpoints, manage own data
`ADMIN` (universal)	Everything RIDER can do, plus: change any user's role

- **Default role on registration:** `RIDER` (Uber's M1 default)
- `ADMIN` is never assigned on register
- Only an `ADMIN` can change another user's role via `PUT /api/users/{id}/role`
- The JWT token's `role` claim contains the user's actual role value
- Seed at least one ADMIN user so role management can be tested

5.4 Security Configuration (Per Service)

Each of the 5 services must have:

- A security configuration with a JWT filter that intercepts every request
- Stateless session management (no server-side sessions)
- CSRF disabled (since this is a token-based API)
- Public endpoints: `/api/auth/register` and `/api/auth/login` (on User Service only), and health checks
- All other endpoints require a valid JWT token

5.5 Password Handling

The User entity already has a `password` field from M1. In M2, you must ensure:

- Passwords are hashed with BCrypt before being stored
- The plaintext password is **never** stored or returned in API responses
- Login verifies the provided password against the stored hash

6 Database Architecture

6.1 Six Databases (Versions Match Task 2 Exactly)

All teams use the same 6 databases. **Docker image versions match Task 2** so students who completed Task 2 can reuse their stack without reconfiguration.

Database	Port	Docker Image	Architectural Role
PostgreSQL	5432	`postgres:17`	Primary relational data (from M1). PG 17 mandatory — PG 18 breaks Hibernate
MongoDB	27017	`mongo:latest`	Activity logs, event history, analytics — used by all services
Redis	6379	`redis:latest`	Caching layer for read-heavy endpoints — used by all services
Elasticsearch	9200	`elasticsearch:8.19.12`	Full-text search across drivers (S2). Pinned version.
Neo4j	7687	`neo4j:latest`	Recommendation graph from riding patterns (S3)
Cassandra	9042	`cassandra:latest`	Time-series location/GPS tracking events (S4)

6.2 Memory Limitations (Required)

Running 6 databases + 5 Spring Boot services strains laptop RAM. Apply these memory limits to avoid system freezes:

Service	Memory Limit	Rationale
Redis	256 MB max, `allkeys-lru` eviction	Bounded cache
Elasticsearch	512 MB JVM heap (`ES_JAVA_OPTS=-Xms512m -Xmx512m`)	Default heap is 4 GB
Cassandra	512 MB heap (`MAX_HEAP_SIZE=512M`, `HEAP_NEWSIZE=128M`)	Default is 1–2 GB
Neo4j	512 MB heap (`NEO4J_server_memory_heap_max__size=512m`)	Default can exceed 1 GB
PostgreSQL, MongoDB	Default	Well-behaved at defaults

Estimated total memory for the stack: PostgreSQL (∼250 MB) + MongoDB (∼400 MB) + Redis (256 MB capped) + Cassandra (∼700 MB) + Neo4j (∼600 MB) + Elasticsearch (∼700 MB) + 5 Spring Boot apps (5×300 MB = 1.5 GB) = ∼**4.5 GB** for the stack alone.

6.3 Dependency Model

Following the pattern from Lab 5 and Task 2:

- **PostgreSQL:** Hard dependency — the service will not start without it
- **MongoDB, Redis, Elasticsearch, Neo4j, Cassandra:** Soft dependencies — if any of these are unavailable, the features that depend on them should degrade gracefully (try-catch), but the service should still start and serve PostgreSQL-based endpoints

6.4 Docker Compose Additions

Add these containers to your existing `docker-compose.yaml` alongside PostgreSQL and the 5 application services:

```
mongo:
  image: mongo:latest
  container_name: uber-mongo
  ports:
    - "27017:27017"
  environment:
    MONGO_INITDB_ROOT_USERNAME: root
    MONGO_INITDB_ROOT_PASSWORD: rootpass
    MONGO_INITDB_DATABASE: ubermongo
```

```
  volumes:
    - mongo-data:/data/db
  healthcheck:
    test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
    interval: 10s
    timeout: 5s
    retries: 5

redis:
  image: redis:latest
  container_name: uber-redis
  ports:
    - "6379:6379"
  command: redis-server --requirepass redispass --maxmemory 256mb --maxmemory-policy allkeys-lru
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "redispass", "ping"]
    interval: 5s
    timeout: 5s
    retries: 5

elasticsearch:
  image: elasticsearch:8.19.12
  container_name: uber-elasticsearch
  ports:
    - "9200:9200"
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  volumes:
    - es-data:/usr/share/elasticsearch/data
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:9200/_cluster/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 30s

neo4j:
  image: neo4j:latest
  container_name: uber-neo4j
  ports:
    - "7474:7474"
    - "7687:7687"
  environment:
    NEO4J_AUTH: neo4j/neo4jpass
    NEO4J_server_memory_heap_max__size: 512m
    NEO4J_server_memory_heap_initial__size: 256m
  volumes:
    - neo4j-data:/data
  healthcheck:
    test: ["CMD", "neo4j", "status"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 30s

cassandra:
  image: cassandra:latest
  container_name: uber-cassandra
  ports:
    - "9042:9042"
```

```
  environment:
    CASSANDRA_CLUSTER_NAME: ubercluster
    CASSANDRA_DC: datacenter1
    CASSANDRA_KEYSPACE: uberks
    MAX_HEAP_SIZE: 512M
    HEAP_NEWSIZE: 128M
  volumes:
    - cassandra-data:/var/lib/cassandra
  healthcheck:
    test: ["CMD-SHELL", "cqlsh -e 'DESCRIBE KEYSPACES'"]
    interval: 15s
    timeout: 10s
    retries: 10
    start_period: 60s

volumes:
  mongo-data:
  es-data:
  neo4j-data:
  cassandra-data:
```

6.5 Reference application.yml Fragments

Each of your 5 services needs the connection properties for every database it uses. Note: M1 used `application.properties`. In M2 you should migrate to `application.yml` (YAML format is what the auto-grader expects).

Shared across all services

```
spring:
  datasource:
    url: jdbc:postgresql://postgres:5432/uberdb
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  data:
    redis:
      host: redis
      port: 6379
      password: redispass

jwt:
  secret: "<Base64-encoded shared secret across all 5 services>"
  expiration: 86400000 # 24 hours in ms
```

All services — MongoDB (event logging)

```
spring:
  data:
    mongodb:
      uri: mongodb://root:rootpass@mongo:27017/ubermongo?authSource=admin
```

Driver service only — Elasticsearch

```
spring:
  elasticsearch:
    uris: http://elasticsearch:9200
```

Ride service only — Neo4j

```
spring:
  data:
    neo4j:
      uri: bolt://neo4j:7687
      username: neo4j
      password: neo4jpass
```

Location service only — Cassandra

```
spring:
  cassandra:
    contact-points: cassandra
    port: 9042
    local-datacenter: datacenter1
    keyspace-name: uberks
    schema-action: CREATE_IF_NOT_EXISTS
```

7 New Entity Models

Your existing PostgreSQL entities from M1 remain unchanged except for the password handling described in Section 4.1 and Section 5.5. This section defines the **new** entities for the NoSQL databases.

7.1 MongoDB Document Entities (All Services)

Each service defines one MongoDB document class for its activity/event log. MongoDB entities must be **classes** (not records).

7.1.1 Common MongoEvent Interface

All 5 MongoDB event classes below implement a shared interface `MongoEvent` so that the `EventFactory` (Section 3.7) can return any concrete event typed as `MongoEvent`:

Method	Return type	Purpose
`getId()`	`String`	MongoDB ObjectId
`getTimestamp()`	`LocalDateTime`	When the event occurred
`getAction()`	`String`	The action identifier (e.g., REGISTERED)
`getDetails()`	`Map<String, Object>`	Additional event context

Each concrete event class below adds its own service-specific fields on top of this common interface.

7.1.2 AuthEvent (User Service)

Collection: `auth_events`

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
userId	Long	not null	References User in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	When the event occurred
details	Map<String, Object>		Additional event context

action **primary values:** REGISTERED, LOGGED_IN, ROLE_CHANGED. **Non-exhaustive** — extend with domain-appropriate values when the Observer retrofit (Section 4.5) writes events from M1 user endpoints, e.g., USER_UPDATED (S1-F2 preferences), USER_DEACTIVATED (S1-F4), DEFAULT_ADDRESS_SET (S1-F7), USER_CREATED / USER_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.3 DriverEvent (Driver Service)

Collection: driver_events

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
driverId	Long	not null	References Driver in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
details	Map<String, Object>		Fields changed, dashboard params, etc.

action **primary values:** INDEXED, UPDATED, DASHBOARD_VIEWED. **Non-exhaustive** — extend for M1 retrofits, e.g., VEHICLE_DETAILS_UPDATED (S2-F2), AVAILABILITY_UPDATED (S2-F4), RATING_RECORDED (S2-F7), DOCUMENT_VERIFIED (S2-F8), DRIVER_CREATED / DRIVER_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.4 RideEvent (Ride Service)

Collection: ride_events

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
rideId	Long	not null	References Ride in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
details	Map<String, Object>		

action **primary values:** ANALYTICS_VIEWED, INTERACTION_RECORDED. **Non-exhaustive** — extend for M1 retrofits, e.g., DRIVER_ASSIGNED (S3-F2), RIDE_COMPLETED (S3-F4), RIDE_CANCELLED (S3-F7), STOPS_ADDED (S3-F8), RIDE_CREATED / RIDE_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.5 LocationEvent (Location Service)

Collection: location_events

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
driverId	Long	not null	References the Driver whose location is being tracked
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
details	Map<String, Object>		

action **primary values:** TRACKING_RECORDED, ANALYTICS_VIEWED. **Non-exhaustive** — extend for M1

retrofits, e.g., LOCATION_UPDATED (S4-F2), BATCH_LOCATIONS_UPDATED (S4-F4), OLD_LOCATIONS_PURGED (S4-F7), LOCATION_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.6 PaymentAuditEvent (Payment Service)

Collection: payment_audit_trail

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
paymentId	Long	not null	References Payment in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
method	String	conditional	See method/amount note below
amount	Double	conditional	See method/amount note below
details	Map<String, Object>		Refund reason, fee breakdown, etc.

action **primary values:** CREATED, COMPLETED, FAILED, REFUNDED, REFUND_DENIED, ANALYTICS_VIEWED. **Non-exhaustive** — extend for M1 retrofits, e.g., COUPON_APPLIED (S5-F5), RETRY_ATTEMPTED (S5-F7), PAYMENT_DELETED (CRUD). Use UPPER_SNAKE_CASE.

method **values:** CREDIT_CARD, CASH, WALLET — matches the M1 Payment.method enum values.

When each action value is written:

- CREATED — written by M1 S5-F4 (Process Payment) when the Payment row is first inserted.
- COMPLETED — written by M1 S5-F4 when the payment status transitions to COMPLETED.
- FAILED — written by M1 S5-F4 when a simulated gateway failure occurs. Since M1 does not model a real payment gateway, enable failure simulation via an optional query parameter ?simulateFailure=true on the S5-F4 create-payment request: when set, S5-F4 short-circuits to Payment.status = FAILED in PG and writes a FAILED audit event. This affordance exists solely to let the S5-F11 test scenario produce failed-payment data; it is not a production flow.
- REFUNDED — written by M1 S5-F2 (simple refund) and by M2 S5-F12 (refund with surge handling) when the refund is successfully processed.
- REFUND_DENIED — written by M2 S5-F12 when NoRefundStrategy blocks the refund (e.g., the refund window has expired). The event records the denial reason and strategy name in details.
- ANALYTICS_VIEWED — written by M2 S5-F10 (Fare Revenue by Vehicle Type) each time the endpoint is invoked, regardless of whether the response is served from cache. Used to track analytics access patterns.

method **and** amount **field requirement:** The fields method and amount are **required (not null)** on every event whose action is a payment-shaped action, i.e., action ∈ {CREATED, COMPLETED, FAILED, REFUNDED, REFUND_DENIED} and any payment-lifecycle retrofit action (e.g., COUPON_APPLIED, RETRY_ATTEMPTED). They are **omitted (null-permitted)** only on non-payment actions such as ANALYTICS_VIEWED (an analytics view is not tied to a specific Payment row's method/amount). S5-F11 (Payment Method Breakdown) relies on this: it groups and sums only events whose method and amount are populated, so a CREATED/COMPLETED/FAILED event written without method would silently vanish from the breakdown.

7.2 Elasticsearch Document (Driver Service)

7.2.1 DriverSearchDocument

Index: drivers

Field	ES Field Type	Purpose
`id`	Keyword	Exact match, corresponds to PG id
`name`	Text (analyzed)	Full-text search on driver name
`vehicleType`	Keyword	Filtering (`SEDAN, SUV, HATCHBACK, VAN, LUXURY` — derived from `vehicleDetails.vehicleType`)
`description`	Text (analyzed)	Full-text search on description (see note below)
`rating`	Double	Range filtering
`status`	Keyword	Filtering (`AVAILABLE, BUSY, OFFLINE` — matches M1 Driver.status enum)

Note on `description`: The M1 Driver.vehicleDetails JSONB schema does not include a `description` key. M2 **additively extends** the M1 JSONB schema with a `description` key holding a free-text driver/vehicle description. Existing M1 records may have this field empty or missing — S2-F11 should default to an empty string when absent.

7.3 Neo4j Entities (Ride Service)

7.3.1 UserNode

Property	Type	Notes
`userId`	`Long`	Corresponds to User.id in PG
`name`	`String`	User's display name

7.3.2 DriverNode

Property	Type	Notes
`driverId`	`Long`	Corresponds to Driver.id in PG
`name`	`String`	Driver's display name
`vehicleType`	`String`	For filtering recommendations

7.3.3 RODE_WITH Relationship

Connects a `UserNode` to a `DriverNode`. Direction: `(User)-[:RODE_WITH]->(Driver)`.

Property	Type	Notes
`rideCount`	`Integer`	Incremented each time this user rides with this driver
`lastRideDate`	`LocalDateTime`	Updated on each ride

7.4 Cassandra Entity (Location Service)

7.4.1 LocationTrackingEvent

Table: `location_tracking_events`

Column	Type	Key	Notes
`driver_id`	`Long`	Partition Key	Groups all location events for one driver
`timestamp`	`Timestamp`	Clustering (DESC)	Orders events newest-first
`latitude`	`Double`		GPS latitude
`longitude`	`Double`		GPS longitude
`speed`	`Double`		Optional — km/h (derived from M1 `Location.metadata.speed`)
`heading`	`Double`		Optional — degrees (derived from M1 `Location.metadata.heading`)
`accuracy`	`Double`		Optional — GPS accuracy in meters (derived from M1 `Location.metadata.accuracy`)
`ride_id`	`Long`		Optional — set when the driver is currently on an active ride, null otherwise
`notes`	`String`		Optional event notes

Queries on this table **must** include `driver_id` in the WHERE clause (partition key requirement).

8 Caching Strategy

All read-heavy endpoints (both new M2 features and existing M1 endpoints) must be cached in Redis.

8.1 TTL Guidelines

Data Type	TTL	Examples
Dashboards / analytics	10 minutes	Revenue dashboard, performance dashboard
Search results	5 minutes	Driver search, recommendations
Activity feeds	5 minutes	User activity feed
Entity detail views	15 minutes	Driver profile, ride details

8.2 Invalidation

Write operations (POST, PUT, DELETE) must invalidate any cached data they affect. The canonical invalidation enumeration for all M1 write endpoints + CRUD writes + M2 observer writes lives in Section 4.4.4 and uses the wildcard-deletion strategy described in Section 4.4.6. Features that mention invalidation (e.g., S5-F12 step h) refer to those rules. In brief:

- Entity write on `<entity>{id}` → delete `<service>::<entity>::{id}` plus wildcard-delete `<service>::S{n}-F{m}::*` for every feature whose cached output may reference that entity.
- M2 analytics events written by Observer retrofit (see Section 4.4.4 M2-writes subsection) also invalidate the matching dashboard caches.
- Over-invalidation (clearing a few cache keys that did not strictly need to be cleared) is acceptable — correctness beats cache-hit optimality at M2.

9 Cross-Cutting Requirements

These requirements apply globally and are **not** counted as numbered features. Each requirement has a dedicated test scenario below.

9.1 CC-1 — JWT on All Endpoints

Requirement: Every endpoint (including your M1 endpoints) must require a valid JWT token in the `Authorization` header, except `POST /api/auth/register`, `POST /api/auth/login`, and health checks. Missing or invalid token returns **401**. Insufficient role returns **403**.

Test scenario:

a) Enumerate every endpoint in all 5 services by scanning controllers. For each: call without `Authorization` header → expect 401 unless the endpoint is `/api/auth/register`, `/api/auth/login`, or a health check.

b) For each public endpoint: call without a token → expect a 2xx response (not 401).

c) Call any protected endpoint with a malformed token (`Bearer abc`) → 401.

d) Call any protected endpoint with an expired token → 401.

e) Call `PUT /api/users/{id}/role` with a valid RIDER token → 403.

f) Count the protected vs public endpoints. Uber should have exactly 3 public endpoints (register, login, health). Anything else public is a grading failure.

9.2 CC-2 — Role Management

Requirement: Implement `PUT /api/users/{id}/role` as an ADMIN-only endpoint that changes a user's role. Must log a `ROLE_CHANGED` event to `auth_events`.

Request body: `{"role": "ADMIN"}` (or `"RIDER"`).

Behavior:

a) Validate JWT and `role` claim is `ADMIN` — ***throws 403 if not ADMIN***.

b) Find user by ID — ***throws 404 if user not found***.

c) Validate requested role is a valid ENUM value — ***throws 400 if invalid***.

d) Update the user's role and save.

e) Log `ROLE_CHANGED` to `auth_events` with the old and new role in `details`.

f) Invalidate cached user detail. **Note:** The `ROLE_CHANGED` write to `auth_events` automatically triggers Â§4.4.4 invalidation of `user-service::S1-F12::*` (the activity feed) via the Observer chain; no extra code is needed in this endpoint.

g) Return the updated user with status ***code 200***.

Token staleness after role change (accepted limitation): The target user's existing JWT continues to carry its **old** `role` claim until it expires. Since tokens have a 24-hour lifetime (see Â§5.2) and the spec does not require a token-revocation list, a demoted user (ADMIN → RIDER) retains ADMIN authorization for up to 24 hours, and a promoted user (RIDER → ADMIN) cannot exercise new ADMIN privileges until they log in again. This is an accepted limitation for M2. The `ROLE_CHANGED` audit event and the invalidated user-detail cache give operators the information they need, but do not revoke the in-flight JWT. Production-grade solutions (token revocation list, shorter expiry + refresh tokens, per-request PG role re-check) are out of scope for M2.

Test scenario:

a) Seed an ADMIN and a RIDER. Log in as ADMIN.

b) `PUT /api/users/{riderId}/role` with body `{"role":"ADMIN"}` → 200, the target user's role is now ADMIN in PG.

c) Verify the `auth_events` collection has a `ROLE_CHANGED` event with `details` containing the old and new roles.

d) Verify the cached user detail (`user-service::user::{riderId}`) was removed from Redis.

e) Verify cached keys under `user-service::S1-F12::{riderId}::*` were removed from Redis after the `ROLE_CHANGED` write propagated through the Observer chain (Â§4.4.4 NoSQL-writer invalidation).

f) Log in as a different RIDER and call `PUT /api/users/1/role` → 403.

g) Call `PUT /api/users/999/role` with ADMIN token → 404.

h) Call with body `{"role":"BANANA"}` → 400.

i) Call without Authorization → 401.

9.3 CC-3 — Redis Caching on M1 Endpoints

Requirement: Your M1 feature GET endpoints (27 for Uber — see Section 4.4.1) and CRUD `GET /api/<entity>/{id}` endpoints (10 for Uber) must be cached per Section 4.4. List endpoints are **not** cached.

Test scenario: See Section 4.4.6 for the comprehensive cache test scenario. Additional spot checks:

a) For each of the 27 cached M1 feature endpoints and 10 CRUD GET-by-ID endpoints: call twice, inspect Redis between calls, confirm a key with the expected `<service>::<featureId>::<param-hash>` or `<service>::<entity>::<id>` format exists.

b) For every M1 write endpoint (the 18 feature writes + 30 CRUD writes): trigger the write, then query Redis and verify the affected detail cache key is gone and relevant feature-result caches are invalidated.

c) Confirm `GET /api/<entity>` (list) endpoints produce NO cache entry.

9.4 CC-4 — Design Pattern Implementation

Requirement: The 7 design patterns defined in Section 3 must be implemented at their specified locations.

Test scenario: See each pattern's own test scenario in Section 3.2 through 3.8.

9.5 CC-5 — Docker Compose with 6 Databases

Requirement: Your `docker-compose.yaml` must include all 6 database containers (PostgreSQL, MongoDB, Redis, Elasticsearch, Neo4j, Cassandra) with versions matching Task 2 alongside your 5 application services.

Test scenario:

a) Parse the team's `docker-compose.yaml`. Assert services named `postgres`, `mongo`, `redis`, `elasticsearch`, `neo4j`, `cassandra` exist.

b) Assert image tags: `postgres:17`, `elasticsearch:8.19.12`, and `mongo:latest`/`redis:latest`/`neo4j:latest`/`cassandra:latest`.

c) Assert memory caps: Redis command includes `-maxmemory 256mb -maxmemory-policy allkeys-lru`; Elasticsearch env has `ES_JAVA_OPTS=-Xms512m -Xmx512m`; Cassandra env has `MAX_HEAP_SIZE: 512M`; Neo4j env has `NEO4J_server_memory_heap_max__size: 512m`.

d) Assert healthchecks exist for all databases.

e) Run `docker compose up -d` → all 6 databases reach healthy state within 120 seconds.

f) All 5 application services start successfully and connect to their databases.

g) Total stack memory usage (`docker stats`) stays under 5 GB.

9.6 CC-6 — Application Configuration (application.yml)

Requirement: Each service's `application.yml` must include connection settings for the databases it uses, plus the shared JWT secret and expiration. Must be YAML format, not properties.

Test scenario:

a) Assert every service has an `application.yml` file (not `application.properties`).

b) For user-service, driver-service, ride-service, location-service, payment-service: assert `spring.datasource.url` is present and points to `postgres:5432`.

c) Assert `spring.data.mongodb.uri`, `spring.data.redis.host`, and `jwt.secret`/`jwt.expiration` are present in every service.

d) Driver-service: assert `spring.elasticsearch.uris` is present.

e) Ride-service: assert `spring.data.neo4j.uri` is present.

f) Location-service: assert `spring.cassandra.contact-points` and `keyspace-name` are present.

g) Boot each service in isolation with other services down → must start as long as PostgreSQL is up (hard dep). NoSQL unavailability must not prevent startup.

10 Features

Milestone 2 adds **15 new features** (3 per service). Feature numbering continues from M1 (which used F1–F9), so M2 features are **F10–F12** per service.

Every M2 feature touches at least 2 databases. Each feature specification lists which databases are involved, whether authentication is required, and the caching behavior.

Important Note: Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.

10.1 User Service Features (port 8081)

10.1.1 [S1-F10] Register User

Branch: `feat/user/S1-F10/<ID>`
Endpoint: `POST /api/auth/register`
Auth: None (public endpoint)
Databases: PostgreSQL, MongoDB
Request body:

```
{
  "name": "Ahmed Ali",
  "email": "ahmed@example.com",
  "password": "securePassword123",
  "phone": "+201012345678"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn": 86400000
}
```

Behavior:

a) Validate that name, email, password, and phone are not blank –***throws 400 if any field is missing or blank***.

b) Check that no user with this email or phone already exists –***throws 409 if the email or phone is already registered*** (the M1 User entity requires both email and phone to be unique).

c) Hash the password before saving with **BCrypt** (the stored password must not be the plaintext value).

d) Create the user in PostgreSQL with role `RIDER` (Uber's M1 default role — never assign `ADMIN` on register) and status `ACTIVE`.

e) Log a `REGISTERED` event to the `auth_events` collection in MongoDB with the user's ID and the current timestamp.

f) Generate a token containing the user's email (as the `sub` claim), the user's numeric `User.id` (as a custom `uid` claim), and the user's role (as a `role` claim). The `uid` claim is required by the ownership checks in Â§10 (S1-F12, S3-F12); see Â§5.2 for the full payload contract.

g) Return the token and expiration with status ***code 201***.

Test scenario:

a) `POST /api/auth/register` with valid data → 201 with token. Verify user exists in database.

b) `POST /api/auth/register` with the same email → 409.

c) `POST /api/auth/register` with the same phone (but different email) → 409.

d) `POST /api/auth/register` with blank email or blank phone → 400.

e) Verify the stored password is **not** the plaintext password (it should be a BCrypt hash).

10.1.2 [S1-F11] Login

Branch: `feat/user/S1-F11/<ID>`
Endpoint: `POST /api/auth/login`
Auth: None (public endpoint)
Databases: PostgreSQL, MongoDB
Request body:

```
{
  "email": "ahmed@example.com",
  "password": "securePassword123"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn": 86400000
}
```

Behavior:

a) Find user by email in PostgreSQL –***throws 401 if user not found***.

b) Verify the provided password against the stored hash –***throws 401 if the password does not match***.

c) Log a `LOGGED_IN` event to the `auth_events` collection in MongoDB with the user's ID and timestamp.

d) Generate a token containing the user's email (as the `sub` claim), the user's numeric `User.id` (as a custom `uid` claim), and the user's role (as a `role` claim). The `uid` claim is required by the ownership checks in Â§10 (S1-F12, S3-F12); see Â§5.2 for the full payload contract.

e) Return the token and expiration with status ***code 200***.

Note on 401 for both error cases: Returning **401** for both "user not found" and "wrong password" is intentional — it prevents account enumeration. A caller cannot distinguish whether a given email is registered. Do **not** return 404 for missing email.

Test scenario:

a) Register a user, then `POST /api/auth/login` with correct credentials → 200 with token.

b) `POST /api/auth/login` with wrong password → 401.

c) `POST /api/auth/login` with non-existent email → 401 (not 404 — this is deliberate to prevent account enumeration).

d) Use the returned token to access a protected endpoint (e.g., `GET /api/users/1`) → should succeed. Access the same endpoint without a token → 401.

10.1.3 [S1-F12] Get User Activity Feed

Branch: `feat/user/S1-F12/<ID>`
Endpoint: `GET /api/users/{id}/activity?page={page}&size={size}`
Auth: Required (USER)
Databases: MongoDB, PostgreSQL, Redis
Response: Paginated list of activity events:

```
{
  "content": [
    {
      "action": "LOGGED_IN",
      "timestamp": "2026-04-20T14:30:00",
      "details": {}
    },
    {
      "action": "REGISTERED",
      "timestamp": "2026-04-20T14:25:00",
      "details": {"email": "ahmed@example.com"}
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 2
}
```

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) **Ownership check:** Verify the authenticated caller's `uid` claim from the JWT equals the path `{id}`, OR the caller's `role` claim is `ADMIN` –***throws 403 if the caller is neither the target user nor an ADMIN*** (prevents account enumeration and PII leakage). The comparison is a direct numeric equality check on the JWT's `uid` claim — no PG lookup needed.

c) Find user by ID in PostgreSQL –***throws 404 if user not found***.

d) Query the `auth_events` collection in MongoDB for events belonging to this user, sorted by most recent first, paginated by the `page` and `size` parameters. **Defaults and bounds:** if `page` is omitted, default to `0`; if `size` is omitted, default to `10`; cap `size` at `100` — requests with `size > 100` are silently clamped to 100.

e) The response should be cached for 5 minutes. Subsequent identical requests within the cache window should return the cached version.

f) Return the paginated activity feed with status ***code 200***.

Note: The activity feed should also include events from role management operations. The role management endpoint (`PUT /api/users/{id}/role`, see Section 9 cross-cutting requirement CC-2) must also be implemented, and `ROLE_CHANGED` events should appear in this feed.

Test scenario:

a) Register a user, login, then `GET /api/users/{id}/activity` with the user's own token → should contain `REGISTERED` and `LOGGED_IN` events ordered by most recent first.

b) Using the seeded ADMIN token, trigger a role change via `PUT /api/users/{id}/role` changing the user's role from RIDER to ADMIN. Then `GET /api/users/{id}/activity` with the affected user's token → should contain a `ROLE_CHANGED` event as the most recent entry.

c) Register user A and user B. With user A's token, call `GET /api/users/{B.id}/activity` → 403 (ownership violation, not 404 — A's token is valid and B exists).

d) With the seeded ADMIN token, call `GET /api/users/{any-user-id}/activity` → 200 (ADMIN bypasses ownership check).

e) `GET /api/users/{id}/activity` without Authorization header → 401.

f) `GET /api/users/999/activity` with an ADMIN token → 404 (ADMIN passes ownership, user not found).

g) Verify pagination: create multiple events, request with `page=0&size=1` (with matching-owner token) → should return 1 event with correct total count.

10.2 Driver Service Features (port 8082)

10.2.1 [S2-F10] Full-Text Driver Search

Branch: `feat/driver/S2-F10/<ID>`
Endpoint:
`GET /api/drivers/search/full-text?query={text}&vehicleType={type}&status={s}&minRating={n}&maxRating={n}`
Auth: Required (USER)
Databases: Elasticsearch, PostgreSQL, Redis
Response: List of matching drivers with relevance-based ordering.

Note on path: This is a **distinct endpoint** from M1's S2-F1 (`GET /api/drivers/search`). M1's endpoint searches PostgreSQL by status and rating range and is unchanged; M2's new `/full-text` endpoint searches Elasticsearch on name and description. Both endpoints must coexist.

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Search drivers using full-text search on the `query` parameter, matching against the driver's name and description. The search should handle partial matches and be case-insensitive.

c) Optionally filter by `vehicleType` (exact match, one of `SEDAN, SUV, HATCHBACK, VAN, LUXURY`), `status` (exact match, one of `AVAILABLE, BUSY, OFFLINE`), and `minRating`/`maxRating` (range filter). All filter parameters are optional — if not provided, they are ignored.

d) Results should be sorted by relevance to the search query.

e) The response should be cached for 5 minutes.

f) Return the matching drivers with status ***code 200***. Return an empty list if no matches.

Test scenario:

a) Create 3 drivers via CRUD and index them. Search with `query=toyota` → should return drivers with "toyota" in name or description.

b) Search with `query=toyota&vehicleType=SEDAN` → should return only SEDAN drivers.

c) Search with `query=toyota&status=AVAILABLE` → should return only AVAILABLE drivers.

d) Search with `query=toyota&minRating=4.0` → should return only highly-rated drivers.

e) Search with `query=nonexistent` → empty list.

f) Search without token → 401.

10.2.2 [S2-F11] Index Driver for Search

Branch: `feat/driver/S2-F11/<ID>`
Endpoint: `POST /api/drivers/{id}/index`
Auth: Required (USER)
Databases: PostgreSQL, Elasticsearch, MongoDB
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Find driver by ID in PostgreSQL –***throws 404 if driver not found***.

c) Read the `description` key from the driver's `vehicleDetails` JSONB field; if the key is missing or null, default to an empty string. (M2 additively extends the M1 JSONB with a `description` key — see Section 7.2 note.) Also read the `vehicleType` key from `vehicleDetails` for the ES `vehicleType` field.

d) Create or update the driver's search document in Elasticsearch with the driver's `id`, `name`, `vehicleType` (from `vehicleDetails.vehicleType`), `description` (from step c), `rating`, and `status`.

e) Log an `INDEXED` event to the `driver_events` collection in MongoDB via the Observer chain. The event's `details` object must include `driverId`, the list of `indexedFields` written in the prior step, and a `source` key: `''explicit''` when invoked through this endpoint, `''auto_crud_create''` / `''auto_crud_update''` when triggered by the CRUD auto-index retrofit at step f. On DELETE the retrofit emits `DRIVER_DELETED` (not `INDEXED`) after removing the ES document.

f) **Auto-index on CRUD changes:** In addition to this explicit endpoint, the driver must be automatically re-indexed whenever it is created or updated via any M1 CRUD endpoint, and **removed from the search index** whenever it is deleted via `DELETE /api/drivers/{id}`. This is graded — tests will create/update a driver via CRUD and then search for it without calling this endpoint explicitly; after a delete, searching for the deleted driver must return no match.

g) Return status ***code 200*** indicating successful indexing.

Test scenario:

a) Create a driver via CRUD with `vehicleDetails.description=''luxury toyota camry''`, then `POST /api/drivers/{id}/index` → 200. Search with `query=toyota` → should return the driver.

b) Create a driver via CRUD whose `vehicleDetails` JSONB has no `description` key → indexing succeeds (empty description).

c) `POST /api/drivers/999/index` → 404.

d) Update a driver's name via CRUD (without calling `/index`), then search with the new name → should find the updated driver (auto-indexing verification).

e) Index without token → 401.

10.2.3 [S2-F12] Get Driver Performance Dashboard

Branch: `feat/driver/S2-F12/<ID>`
Endpoint: `GET /api/drivers/{id}/dashboard`
Auth: Required (USER)
Databases: PostgreSQL, MongoDB, Redis
Response DTO (`DriverDashboardDTO`): `driverId`, `name`, `totalRides`, `totalEarnings`, `averageRideFare`, `averageRating`, `totalRatings`.
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Find driver by ID in PostgreSQL –***throws 404 if driver not found***.

c) Aggregate from the `rides` and `payments` tables (joined on `payments.ride_id = rides.id`): total number of COMPLETED rides for this driver, total earnings (sum of `payments.amount` on completed rides), average ride fare.

d) Read the driver's current `rating` and `totalRatings` columns from the `drivers` table.

e) Log a `DASHBOARD_VIEWED` event to the `driver_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Â§4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.

f) The response should be cached for 10 minutes.

g) Return the dashboard DTO with status ***code 200***.

Test scenario:

a) Create a driver with rating 4.5 and 100 totalRatings. Create 5 COMPLETED rides for this driver with payment amounts 100, 200, 150, 300, 250. `GET /api/drivers/{id}/dashboard` → totalRides=5, totalEarnings=1000, averageRideFare=200, averageRating=4.5, totalRatings=100.

b) `GET /api/drivers/999/dashboard` → 404.

c) Driver with no rides → totalRides=0, totalEarnings=0.

d) Without token → 401.

10.3 Ride Service Features (port 8083)

Use Builder pattern

10.3.1 [S3-F10] Get Ride Analytics Dashboard

Branch: `feat/ride/S3-F10/<ID>`
Endpoint: `GET /api/rides/analytics/dashboard?startDate={date}&endDate={date}`
Auth: Required (USER)
Databases: PostgreSQL, MongoDB, Redis
Response DTO (`RideAnalyticsDashboardDTO`): `totalRides`, `totalRevenue`, `averageRideFare`,

`completionRate`, `ridesByStatus` (map of status to count).

Note on path and DTO: This is a **distinct endpoint and DTO** from M1's S3-F6. M1's `GET /api/rides/analytics` returns `RideAnalyticsDTO` (totalRides, completedRides, cancelledRides, totalRevenue, averageRideFare, completionRate) and remains unchanged. M2's new `/dashboard` endpoint returns the richer `RideAnalyticsDashboardDTO` with a status breakdown map. Both must coexist — do not overwrite either.

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Validate date range –***throws 400 if startDate is after endDate***. Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range `[startDate T00:00:00, endDate T23:59:59.999]` in the server time zone, matching the PG `TIMESTAMP` columns (`requestedAt`, `completedAt`, etc.) and the MongoDB `timestamp` field. Boundary rows (events exactly at `startDate T00:00:00` or `endDate T23:59:59.999`) are included.

c) Aggregate ride statistics from the `rides` table (joined with `payments` for revenue) for the given date range: total rides, total revenue (sum of `payments.amount` across COMPLETED rides), average ride fare, completion rate (COMPLETED rides divided by total rides), and a breakdown of ride counts by status.

d) Log an `ANALYTICS_VIEWED` event to the `ride_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Â§4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.

e) The response should be cached for 10 minutes.

f) Return the analytics DTO with status ***code 200***.

Test scenario:

a) Create 10 rides in March 2026: 6 COMPLETED, 2 CANCELLED, 2 REQUESTED. `GET /api/rides/analytics/dashboard?startDate=2026-03-01&endDate=2026-03-31` → totalRides=10, completionRate=0.6, ridesByStatus={COMPLETED:6, CANCELLED:2, REQUESTED:2}.

b) Query with dates where no rides exist → all values 0.

c) `startDate=2026-04-01&endDate=2026-03-01` → 400.

d) Without token → 401.

10.3.2 [S3-F11] Record User-Driver Riding Pattern

Branch: `feat/ride/S3-F11/<ID>`
Endpoint: `POST /api/rides/{rideId}/record-interaction`
Auth: Required (USER)
Databases: Neo4j, PostgreSQL, MongoDB
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Find the ride by ID in PostgreSQL –***throws 404 if ride not found***.

c) Verify the ride's status is `COMPLETED` –***throws 400 if the ride is not completed*** (meaning we only record interactions for completed rides).

d) **Idempotency check:** Before mutating the graph, check whether this specific `rideId` has already been recorded. Maintain the idempotency marker **inside Neo4j** (not PostgreSQL — M2 does not alter any M1 PG schema per Â§1). Two acceptable Neo4j-only approaches: (a) a `recorded_ride_ids` collection property on the `RODE_WITH` relationship that accumulates the set of rideIds contributing to the edge's `rideCount`, or (b) a dedicated Neo4j idempotency sentinel node, e.g., `(User)-[:RECORDED_RIDE {rideId:...}]->(Driver)`, checked with an `EXISTS` query. If the marker says this ride was already recorded, **return 200 immediately without mutating rideCount or lastRideDate**. This guarantees the endpoint is idempotent under repeat POSTs without introducing any M1 PostgreSQL schema change.

e) Look up the user who took this ride and the driver who drove it using the M1 cross-service native SQL pattern — query the `users` and `drivers` tables directly via the shared PostgreSQL database using the ride's foreign key columns (`user_id`, `driver_id`). **Do not introduce inter-service HTTP calls**; those come in Milestone 3.

f) In the recommendation graph: find or create a node for the user and a node for the driver. Then find or create a `RODE_WITH` relationship between them. If the relationship already exists, increment the `rideCount` by 1 and update the `lastRideDate`. If it is new, set `rideCount` to 1. Record this `rideId` in the idempotency marker from step d so future calls with the same `rideId` are no-ops.

g) Log an `INTERACTION_RECORDED` event to the `ride_events` collection in MongoDB via the Observer chain. The event's `details` object must include `rideId`, `userId`, and `driverId`. This write is what causes the S3-F12 recommendation cache to be wildcard-invalidated per Â§4.4.4 (NoSQL-writer → cached-reader rule); it is therefore required on the non-idempotent path only — on the idempotent short-circuit at step d, the graph did not change, so do **not** emit this event.

h) Return status ***code 200*** with a confirmation message.

Test scenario:

a) Create a user, a driver, and a completed ride R1. `POST /api/rides/R1/record-interaction` → 200, rideCount=1.

b) Call the same endpoint again on the same R1 → 200, but rideCount is still 1 (idempotency — repeated calls on the same rideId must not inflate the counter).

c) Create a second completed ride R2 from the same user with the same driver. `POST /api/rides/R2/record-interaction` → 200, rideCount=2.

d) Call with a REQUESTED (not completed) ride → 400.

e) Call with non-existent ride ID → 404.

f) Without token → 401.

10.3.3 [S3-F12] Get Driver Recommendations for User

Branch: `feat/ride/S3-F12/<ID>`
Endpoint: `GET /api/rides/recommendations?userId={id}&limit={n}`
Auth: Required (USER)
Databases: Neo4j, PostgreSQL, Redis
Response: List of `DriverRecommendationDTO`: `driverId`, `name`, `vehicleType`, `score` (how many similar users rode with this driver).
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid.***

b) **Ownership check:** Verify the authenticated caller's `uid` claim from the JWT equals the `userId` query parameter, OR the caller's `role` claim is `ADMIN` –***throws 403 if the caller is neither the target user nor an ADMIN*** (recommendations reveal ride-pattern adjacency — exposing them to arbitrary callers leaks behavioral data). The comparison is a direct numeric equality check on the JWT's `uid` claim — no PG lookup needed.

c) Find user by ID in PostgreSQL –***throws 404 if user not found***.

d) Traverse the recommendation graph: find other users who rode with the same drivers as this user (meaning they share at least one driver in common), then find drivers those other users rode with that this user has **not** ridden with yet. Rank the results by how many of those similar users rode with each recommended driver.

e) Enrich the results with driver details (name, vehicle type) from PostgreSQL.

f) Limit results to the top `limit` recommendations. Default `limit` to 5 if not provided.

g) The response should be cached for 5 minutes.

h) Return the recommendations with status ***code 200***. Return an empty list if no recommendations can be made.

Test scenario:

a) Create 3 users (A, B, C) and 4 drivers (D1, D2, D3, D4). Record interactions: A→D1, A→D2; B→D1, B→D3; C→D2, C→D4. Get recommendations for A with A's own token → should include D3 (because B also rode with D1) and D4 (because C also rode with D2). Should NOT include D1 or D2.

b) Get recommendations for A with B's token → 403 (ownership violation).

c) Get recommendations for A with an ADMIN token → 200 (ADMIN bypass).

d) Get recommendations for a user with no recorded interactions (own token) → empty list.

e) `userId=999` with ADMIN token → 404.

f) Without token → 401.

10.4 Location Service Features (port 8084)

10.4.1 [S4-F10] Get Location Analytics Dashboard

Branch: `feat/location/S4-F10/<ID>`
Endpoint: `GET /api/locations/analytics?startDate={date}&endDate={date}`
Auth: Required (USER)
Databases: PostgreSQL, MongoDB, Redis
Response DTO (`LocationAnalyticsDTO`): `totalLocationEvents`, `activeDrivers`, `averageSpeed`, `eventsByHour` (map of hour 0–23 to count).
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Validate date range –***throws 400 if startDate is after endDate***. Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range `[startDate T00:00:00, endDate T23:59:59.999]` in the server time zone, matching the PG `TIMESTAMP` columns (`timestamp`, `createdAt`, etc.) and the MongoDB `timestamp` field. Boundary rows (events exactly at `startDate T00:00:00` or `endDate T23:59:59.999`) are included.

c) Aggregate from the `locations` table for the given date range (using the `timestamp` column).

d) **totalLocationEvents:** count of location rows whose `timestamp` falls within `[startDate, endDate]`.

e) **activeDrivers:** count of distinct `driverId` values in the range — drivers who recorded at least one location event.

f) **averageSpeed:** average of the `metadata->>'speed'` JSONB key across all rows in the range (cast to numeric; treat missing as 0 for the average only if a row has `speed`). Return 0 if no rows.

g) **eventsByHour:** count of events grouped by the hour-of-day extracted from `timestamp` (EXTRACT(HOUR FROM timestamp)). Values 0–23.

h) Log an `ANALYTICS_VIEWED` event to the `location_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Â§4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.

i) The response should be cached for 10 minutes.

j) Return the analytics DTO with status ***code 200***.

Test scenario:

a) Create 8 location events in April 2026 across 3 distinct drivers, with speed values ranging 20–80 km/h, distributed across hours 8 and 17. `GET /api/locations/analytics?startDate=2026-04-01&endDate=2026-04-30` → totalLocationEvents=8, activeDrivers=3, averageSpeed computed from the 8 rows, eventsByHour contains entries for hour 8 and hour 17.

b) Dates with no location events → totalLocationEvents=0, activeDrivers=0, averageSpeed=0, empty eventsByHour.

c) Invalid date range → 400.

d) Without token → 401.

10.4.2 [S4-F11] Record Location GPS Event

Branch: `feat/location/S4-F11/<ID>`
Endpoint: `POST /api/locations/{driverId}/tracking`
Auth: Required (USER)
Databases: Cassandra, PostgreSQL, MongoDB
Request body:

```
{
  "latitude": 30.0444,
  "longitude": 31.2357,
  "speed": 45.0,
  "heading": 180.0,
  "accuracy": 5.0,
  "rideId": 42,
  "notes": "Heading to pickup location"
}
```

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Verify the driver exists by looking up `drivers` table in PostgreSQL –***throws 404 if driver not found***.

c) Store a new tracking event in the Cassandra time-series table with the driver ID as the partition key, the current server timestamp as the clustering column, and the latitude, longitude, speed, heading, accuracy, optional rideId, and optional notes from the request body. Cassandra is the **primary storage** for the location time-series.

d) **Additionally**, fire the Observer chain to log a `TRACKING_RECORDED` event to the `location_events` MongoDB collection with the `driverId`, the coordinates, and a summary of the tracking payload in `details`. Both writes must succeed independently — Cassandra for the time-series trail, MongoDB as the observability audit log that feeds analytics.

e) Return status ***code 201*** indicating the event was recorded.

Test scenario:

a) Create a driver via CRUD. `POST /api/locations/{driverId}/tracking` with latitude/longitude/speed → 201. Verify a row exists in Cassandra `location_tracking_events` for this `driverId`, AND a `TRACKING_RECORDED` document exists in MongoDB `location_events`.

b) Record another event 10 seconds later with different coordinates → 201. Query the timeline (S4-F12) → should show both events in reverse chronological order.

c) `POST /api/locations/999/tracking` → 404.

d) Without token → 401.

10.4.3 [S4-F12] Get Location Tracking Timeline

Branch: `feat/location/S4-F12/<ID>`
Endpoint: `GET /api/locations/{driverId}/tracking?startTime={datetime}&endTime={datetime}`
Auth: Required (USER)
Databases: Cassandra, PostgreSQL, Redis
Response: List of `LocationTrackingDTO`: `timestamp`, `latitude`, `longitude`, `speed`, `heading`, `accuracy`, `rideId`, `notes`.
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Verify the driver exists in PostgreSQL –***throws 404 if driver not found***.

c) Query the Cassandra time-series table for all tracking events for this driver, optionally filtered by the `startTime` and `endTime` parameters. If no time range is provided, return all events. Results are sorted by most recent first (Cassandra clustering order).

d) The response should be cached for 5 minutes.

e) Return the tracking timeline with status ***code 200***. Return an empty list if no events exist.

Test scenario:

a) Create a driver. Record 3 tracking events: at 14:00, 14:15, 14:30 with different coordinates. `GET /api/locations/{driverId}/tracking` → should return 3 events with the 14:30 event first.

b) Query with `startTime=14:10&endTime=14:20` → should return only the 14:15 event.

c) `GET /api/locations/999/tracking` → 404.

d) Driver with no tracking events → empty list.

e) Without token → 401.

10.5 Payment Service Features (port 8085)

10.5.1 [S5-F10] Get Fare Revenue by Vehicle Type with Surge Breakdown

Branch: `feat/payment/S5-F10/<ID>`
Endpoint: `GET /api/payments/analytics/vehicle-type?startDate={date}&endDate={date}`
Auth: Required (USER)
Databases: PostgreSQL, MongoDB, Redis
Response: List of `VehicleTypeRevenueDTO`: `vehicleType`, `baseFareRevenue`, `surgeFeeRevenue`, `totalRevenue`, `rideCount`.

Surge fee source: S5-F10 reads `transactionDetails.surgeFee` written by M1's updated S5-F4. See Section 4.6 for the full specification of this additive JSONB key and the 15%-of-amount fallback rule for pre-M2 Payments that do not carry the key.

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Validate date range –***throws 400 if startDate is after endDate***. Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range `[startDate T00:00:00, endDate T23:59:59.999]` in the server time zone, matching the PG `TIMESTAMP` columns (`requestedAt`, `createdAt`, etc.) and the MongoDB `timestamp` field. Boundary rows (events exactly at `startDate T00:00:00` or `endDate T23:59:59.999`) are included.

c) Use the M1 cross-service native SQL pattern: JOIN `payments` with `rides` (on `rides.id = payments.ride_id`) and `drivers` (on `drivers.id = rides.driver_id`) on the shared PostgreSQL database. **Do not introduce inter-service HTTP calls**; those come in Milestone 3.

d) Filter by `rides.requestedAt` within `[startDate, endDate]` and `payments.status = COMPLETED`.

e) Group by `drivers.vehicleDetails->>'vehicleType'` (extracted from the Driver's JSONB). For each group compute:

- `surgeFeeRevenue` = sum of `transactionDetails->>'surgeFee'` cast to numeric (default to 15% of the payment amount when the key is missing).
- `baseFareRevenue` = sum of `payments.amount` − `surgeFeeRevenue`.
- `totalRevenue` = sum of `payments.amount`.
- `rideCount` = count of distinct ride IDs.

f) Log an `ANALYTICS_VIEWED` event to the `payment_audit_trail` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Â§4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.

g) The response should be cached for 10 minutes.

h) Return the breakdown with status ***code 200***. Return an empty list if no data exists for the date range.

Test scenario:

a) Create drivers: D1 (SEDAN), D2 (SUV), D3 (SEDAN). Create rides and payments: 3 rides on SEDAN drivers totaling 600 (with 90 surge fees), 2 rides on SUV totaling 400 (with 60 surge fees). `GET analytics/vehicle-type` → SEDAN: baseFareRevenue=510, surgeFee=90, total=600, count=3. SUV: baseFareRevenue=340, surgeFee=60, total=400, count=2.

b) Date range with no payments → empty list.

c) Invalid date range → 400.

d) Without token → 401.

10.5.2 [S5-F11] Get Payment Method Breakdown

Branch: `feat/payment/S5-F11/<ID>`
Endpoint: `GET /api/payments/analytics/methods?startDate={date}&endDate={date}`
Auth: Required (USER)
Databases: MongoDB, PostgreSQL, Redis
Response: List of `PaymentMethodDTO`: `method`, `successCount`, `failureCount`, `successRate`, `totalAmount`.
Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Validate date range –***throws 400 if startDate is after endDate***. Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range `[startDate T00:00:00, endDate T23:59:59.999]` in the server time zone, matching the PG `TIMESTAMP` columns (`createdAt`, etc.) and the MongoDB `timestamp` field. Boundary rows (events exactly at `startDate T00:00:00` or `endDate T23:59:59.999`) are included.

c) Query the `payment_audit_trail` collection in MongoDB filtered by `timestamp` within the fully-closed range `[startDate, endDate]` (inclusive on both ends; interpret both as naive `LocalDateTime`, same server time zone as the PG `created_at` columns). Consider only events with `action` ∈ `{COMPLETED, FAILED}` — exclude `CREATED`, `REFUNDED`, `REFUND_DENIED`, and `ANALYTICS_VIEWED` from the method-breakdown counts.

d) Group the filtered events by `method` field (`CREDIT_CARD`, `CASH`, `WALLET`, matching the M1 Payment.method enum). For each method:

- `successCount` = count of events where `action == COMPLETED`
- `failureCount` = count of events where `action == FAILED`
- `successRate` = `successCount / (successCount + failureCount)` (return 0 if denominator is 0)
- `totalAmount` = sum of `amount` across the COMPLETED events only (do not sum failed amounts)

e) The response should be cached for 10 minutes.

f) Return the breakdown with status ***code 200***. Return an empty list if no data.

Test scenario:

a) Create payment audit events: 5 successful CREDIT_CARD payments totaling 500, 2 failed CREDIT_CARD payments, 3 successful CASH payments totaling 300. `GET analytics/methods` → CREDIT_CARD: success=5, failure=2, rate≈0.71, total=500. CASH: success=3, failure=0, rate=1.0, total=300.

b) Date range with no events → empty list.

c) Invalid date range → 400.

d) Without token → 401.

10.5.3 [S5-F12] Process Ride Refund with Surge Handling

Branch: `feat/payment/S5-F12/<ID>`
Endpoint: `POST /api/payments/{id}/refund-surge-adjusted`
Auth: Required (USER)
Databases: PostgreSQL, MongoDB, Redis
Design Pattern: Strategy (see Section 3.2)
Request body:

```
{
  "reason": "driver_no_show",
  "refundSurge": true
}
```

Note on path: This is a **distinct endpoint** from M1's S5-F2 (`PUT /api/payments/{id}/refund`). M1's endpoint performs a simple full refund and remains unchanged; M2's new `/refund-surge-adjusted` endpoint selects a refund strategy at runtime (see Section 3.2) and writes a richer MongoDB audit trail. Both must coexist.

Behavior:

a) Validate the JWT token –***throws 401 if missing or invalid***.

b) Find payment by ID in PostgreSQL –***throws 404 if payment not found***.

c) Validate the payment status is `COMPLETED` –***throws 400 if the payment is not in COMPLETED status*** (you cannot refund a payment that has not been completed).

d) Select the refund strategy (see Section 3.2 for the three required strategies) **without throwing yet**: `FullRefundWithSurgeStrategy` (when `refundSurge=true` and the payment is within the 24-hour window from `createdAt`), `BaseFareOnlyRefundStrategy` (when `refundSurge=false` and within the window), or `NoRefundStrategy` (when the payment is older than 24 hours from `createdAt`).

e) **If the selected strategy is** `NoRefundStrategy`**:** (i) Log a `REFUND_DENIED` event to the `payment_audit_trail` collection in MongoDB with the strategy name `NoRefundStrategy` and the denial reason **''`refund window expired`''**. (ii) Invalidate the S5-F10 and S5-F11 analytics caches (`payment-service::S5-F10::*` and `payment-service::S5-F11::*`) — even though no PostgreSQL row mutates on the denial path, the new `REFUND_DENIED` audit event changes what those analytics would return on the next read. (iii) Then ***throw 400 with message "refund window expired"***. Logging and cache invalidation must both happen **before** the throw so the audit event is persisted and cached reads pick up the new event immediately.

f) Otherwise (strategy is `FullRefundWithSurgeStrategy` or `BaseFareOnlyRefundStrategy`): invoke the strategy's `calculateRefund(payment, request)` to obtain the refund amount. Update the payment status to `REFUNDED`.

g) Record the refund in the `transactionDetails` JSONB column of the Payment (reusing the M1 JSONB pattern from S5-F2). Add keys: `refundAmount`, `refundSurgeIncluded`, `refundReason`, and `refundedAt`. Do **not** add a new SQL column.

h) Log a `REFUNDED` event to the `payment_audit_trail` collection in MongoDB with the strategy name, reason, original amount, refund amount, and whether the surge fee was included.

i) Invalidate `payment-service::S5-F10::*`, `payment-service::S5-F11::*`, and `payment-service::payment::{id}` (per Â§4.4.4 NoSQL-writer + PG-writer rules). Over-invalidation is acceptable per Â§8.2.

j) Return the updated payment with status ***code 200***.

Test scenario:

a) Create a completed payment of 150 (base fare: 130, surge fee: 20), created today. `POST /api/payments/{id}/refund-surge-adjusted` with `refundSurge: true` → 200, refund amount = 150, status = REFUNDED. Verify `transactionDetails.refundAmount=150` and `refundSurgeIncluded=true`.

b) Create another completed payment of 200 (base fare: 170, surge fee: 30), created today. Refund with `refundSurge: false` → 200, refund amount = 170. Verify `transactionDetails.refundAmount=170` and `refundSurgeIncluded=false`.

c) Create a payment with `createdAt` 2 days ago → 400 with message "refund window expired" (NoRefundStrategy selected). Verify that a `REFUND_DENIED` document exists in `payment_audit_trail` for this `paymentId` (audit log is written **before** the 400 is thrown). Also verify that any previously-cached key under `payment-service::S5-F10::*` and `payment-service::S5-F11::*` was removed from Redis (cache invalidation on the denial path).

d) Attempt to refund a PENDING payment → 400.

e) Attempt to refund an already REFUNDED payment → 400.

f) `POST /api/payments/999/refund-surge-adjusted` → 404.

g) Without token → 401.